Empirical Software Engineering. Accepted Manuscript
The publisher's version is available at: https://doi.org/10.1007/s10664-025-10765-4

Maintaining shared understanding of non-functional requirements in small companies using continuous software engineering

Colin Werner · **Laura Okpara** · **Klaas-Jan Stol** · **Daniela Damian**

Received: 18 March 2025 / Accepted: 3 November 2025

Abstract

Non-functional requirements (NFR) such as performance and maintainability are key to overall software quality. To ensure NFRs can be satisfied, it is important that developers and other stakeholders have a shared understanding of such NFRs to prevent divergence of interpretations and expensive rework as a result. This article is part of a research program to study the management of shared understanding of NFRs. We present a process theory comprising six propositions developed through secondary analysis of extensive field investigations of how small software organizations maintain a shared understanding of NFRs in fast-paced Continuous Software Engineering (CSE) contexts. Small organizations, including software startups, represent an important part of the software industry, but are categorically different from large organizations in that they have fewer financial and staff resources. Further, while literature has suggested general practices for building shared understanding, how this applies to NFRs specifically remains an open question. Based on this inductive theory development, the resultant theory explains how a lack of shared understanding is detected and addressed, and how the CSE context induces challenges in these processes.

Keywords Non-functional requirements · Shared understanding · Small software companies · Continuous software engineering · Process theory

Colin Werner
Expeto Wireless

Laura Okpara
Microsoft Xbox

Klaas-Jan Stol
University College Cork, Cork, Ireland E-mail: k.stol@ucc.ie

Daniela Damian
University of Victoria, Victoria, British Columbia, Canada

1 Introduction

There is widespread recognition that satisfying NFRs, such as system availability, code maintainability, and responsiveness, is key to the success of software systems, particularly as they become increasingly complex and distributed (Caracciolo et al., 2014; Glinz, 2007; Mairiza et al., 2010; Wiegers and Beatty, 2013). A lack of attention to NFRs could potentially derail a software project. Consider privacy as an example; systems operating in Europe that violate the General Data Protection Regulation (GDPR) could now be subject to major fines that run into the hundreds of millions euros. The consequences of not implementing such privacy-related regulation are therefore very significant. As another example: the cost of downtime of software systems tends to exceed several million U.S. dollars. A recent report suggests that downtime at the top 2,000 companies costs them $400 billion a year in missed revenue.[1] Clearly, ensuring system reliability and availability pays off. Yet, organizations must frequently make trade-offs between timely delivery of promised software features and rigorous system design that incorporates sufficient attention for NFRs.

Iterative and incremental methods, such as agile methods, have become the norm to shorten the time to release new products and product versions to market. In more recent years, software companies are shifting from agile methods to 'continuous' practices such as Continuous Integration and Continuous Delivery, supported by automation and focused on a more continuous 'flow' of releases, instead of the time-boxed cadence of agile methods. This has led to a trend that has been labelled continuous software engineering (CSE) (Bosch, 2014; Fitzgerald and Stol, 2017; Klotins et al., 2023). CSE relies on automated and fast releases of new versions, delivering new features quickly to users (Gralha et al., 2018; Klotins et al., 2023). However, feature development is often driven by functional requirements, while a focus on fast delivery frequently means that NFRs receive less attention. Previous work has pointed out that NFRs are frequently neglected in agile development (Ramesh et al., 2010), and indeed, little work exists that has explored NFRs in the context of CSE (Behutiye et al., 2017).

A major complication of NFRs is that they relate to an entire system's architecture (Bass et al., 2012), which is problematic for several reasons. First, NFRs are cross-cutting concerns (Bellomo et al., 2014), which makes understanding their implementation and scope challenging. Second, evaluating the impact of frequent updates, which are inherent to a CSE process, on NFRs is very challenging. Third, maintaining a shared understanding of a system's architecture among all developers within a software project can be very challenging. While shared understanding fosters a more effective, motivated, and collaborative team, more efficient use of resources, and reduces conflict (Darch et al., 2009), there is a paucity of research on shared understanding in software engineering projects in general (Glinz and Fricker, 2015), and in particular with NFRs.

As part of a research program focused on managing shared understanding of NFRs in small organizations using CSE, we conducted a series of field studies (Okpara et al., 2022, 2023; Werner et al., 2020, 2022). We focused on small software companies because they are categorically different from large organizations, primarily because they have fewer resources, both financially and in terms of staff (Richardson and Von Wangenheim, 2007). A smaller staff implies a smaller organizational memory, less diverse expertise, and a stronger focus on survival (Sánchez-Gordón and O'Connor, 2016). Small organizations

[1] https://queue-it.com/blog/cost-of-downtime/

are, by definition, those who employ up to 50 employees.[2] While large organizations tend to be better known within the software industry, a large majority of organizations are in fact small and represent an important part of the software industry (Richardson and Von Wangenheim, 2007; Sánchez-Gordón and O'Connor, 2016).

In a first preliminary empirical study, Werner et al. (2020) reported three factors that contribute to a lack of shared understanding of NFRs. Subsequently, we also identified four practices to manage NFRs and the challenges that the CSE context introduces (Werner et al., 2022), and studied how a remote software organization achieves a shared understanding of NFRs (Okpara et al., 2022, 2023). The present article draws on this dataset compiled through these field studies, and revisits, integrates, and expands our earlier findings to address the following research question:

> How do small software organizations maintain a shared understanding of non-functional requirements in fast-paced continuous software engineering contexts?

We adopted an inductive theory development strategy (Stol and Fitzgerald, 2018), drawing on the field studies mentioned above, resulting in a dialectic process theory. Developing theory has a number of advantages. First, a theory becomes a "node for ideas around which knowledge is concentrated" (Miner, 2015, p. xi). Whereas our earlier articles reported on a number of empirical insights, they lack integration into a parsimonious representation of knowledge on shared understanding of NFRs in CSE. A theory helps to establish a cumulative tradition that ties studies together, so as to facilitate a deeper understanding of a phenomenon that goes beyond the idiosyncratic and the 'here and now,' allowing us to make "lasting contributions" (Stol, 2024). Second, good theories instill further research (Miner, 2015), which can advance or challenge the proposed theory. This in turn helps the research field to focus on asking important questions (Stol, 2024). Third, theory development can help to explain what might otherwise seem as 'obvious' findings (Stol, 2024). Theorizing helps to unpack the mechanisms and assumptions underpinning a phenomenon of interest. Theorizing thus requires a deep understanding of prior literature, if any, and any assumptions that might exist, and comparing and juxtaposing those with empirical findings.

Through our theorizing, we developed six propositions that shed light on how small software organizations maintain a shared understanding of NFRs in fast-paced CSE contexts. Together, these propositions represent a dialectic process theory, which we labelled the Iceberg theory, describing how a lack of shared understanding is detected through what we previously have labelled 'triggers' (Werner et al., 2020) (Proposition 1), and how these critical events can lead organizations to respond to this lack of shared understanding by what we call management practices (Proposition 2). Besides building shared understanding of NFRs *in response to* events, we also found that companies use communication practices to *proactively* build shared understanding of NFRs (Proposition 3), and that different NFRs require different practices (Proposition 4). Further, we found that the specific characteristics and context associated with continuous software engineering in small companies can introduce a variety of challenges (Proposition 5). Finally, we observe that the boundary around the state of shared understanding of an NFR is in constant flux (Proposition 6).

In the remainder of this article, we describe the background of the phenomenon we focus on in Section 2, which concludes with a summary of our current understanding

[2] We acknowledge this cut-off value is rather arbitrary; a company with 51 employees is, of course, not categorically different from one with 50 employees.

and open questions. We then present details on our theory development process in Section 3. In Section 4, we develop the six propositions that address key questions surrounding the maintenance, building, and decay of shared understanding of NFRs in CSE contexts. Addressing each key question, we summarize our current understanding, either by highlighting a lack thereof, or what the literature appears to suggest; we then present a proposition that formalizes our answer to that key question, followed by evidence grounded in fieldwork, and offer further explanations of the proposition. Section 5 discusses the validity and evaluation of the Iceberg theory before setting out a number of implications for research and practices. We conclude this article in Section 6.

2 Background

2.1 Continuous Software Engineering

Continuous Software Engineering has emerged as the next evolutionary step from agile methods (Bosch, 2014; Fitzgerald and Stol, 2017; Klotins et al., 2023). Agile methods emerged in the early 1990s, emphasizing iterative and incremental development, feedback, and responsiveness (Tkalich et al., 2025). Several practices, such as Continuous Integration, and later Continuous Deployment and Continuous Delivery, originated as part of agile methods, but have shifted the 'tempo' from agile sprints to a continuous, 'anytime' pace. Fitzgerald and Stol labelled this phenomenon 'continuous *' (Fitzgerald and Stol, 2017), which extends to all practices within the software development lifecycle, including continuous planning, continuous use, and continuous adoption (Barbala et al., 2024). A major shift that differentiates CSE from agile methods is the continuous nature, versus the steady cadence of 2–3-week iterations common in agile methods such as Scrum (Fitzgerald and Stol, 2017; Klotins et al., 2023; Tkalich et al., 2025). As in agile, CSE seeks to deliver new functionality and features rapidly, but unlike agile, CSE does so at a higher pace, "without delay" (Klotins et al., 2023), possibly daily or multiple times per day, unrestricted by time-boxed iterations typical in agile methods. This approach resembles the principle of 'flow' from lean manufacturing (Fitzgerald and Stol, 2017). It does so by leveraging a wide range of tools that emphasize automation and allow an organization to rapidly and efficiently make continuous changes to their software products. Apart from common tools such as version control systems and testing frameworks, other tools to support CSE include CI servers such as Bamboo and Jenkins and configuration tools such as Puppet and Chef (an overview of tools and practices is offered by Shahin et al. (2017)).

The implication of a fully automated CSE delivery pipeline is that code changes are deployed much faster than was the case following an agile approach. This, however, has major consequences for NFRs, which are typically crosscutting in a code base and cannot be easily handled in chunks of work that fit within a development cycle (Bellomo et al., 2014), a topic that we discuss in more detail next.

2.2 Managing Non-Functional Requirements

While the importance of NFRs has been widely acknowledged (Borg et al., 2003; Glinz, 2007; Mairiza et al., 2010), there are a number of issues that make any discussion of NFRs challenging. We discuss these briefly so as to clarify our position.

First, the distinction between functional and non-functional requirements is ambiguous, perhaps even arbitrary. Intuitively, the classification of requirements as being either functional or non-functional makes sense: the former declares *what* the system shall do, and the latter declares *how* the system behaves (Dorfman and Thayer, 1990; Eckhardt et al., 2016, p. 328). In practice, however, this distinction is not so clear (Eckhardt et al., 2016). Glinz (2007) offers an example of a system to process real-time data from a particle accelerator system (which generates petabytes of data): the functional requirement that the system must process the data in real-time is, in fact, a performance requirement; performance is typically considered a non-functional requirement. The ISO/IEC/IEEE 29148 standard on requirements engineering defines non-functional requirements "how a system is supposed to *be*," and under this umbrella term it differentiates between quality requirements (e.g. reliability, maintainability, security), and human factors requirements as those "required characteristics for the outcomes of interaction with human users" (ISO/IEC/IEEE, 2018, p. 14). Eckhardt et al. (2016) found that, in practice, many NFRs are mislabelled as such when they are in fact describing system behavior.

Second, and related to the first point, is the question whether or not the term NFR is appropriate; Glinz (2007) identified major conceptual discrepancies among different definitions of 'NFR.' Some scholars prefer the term quality requirement (Pohl and Rupp, 2016; Svensson et al., 2010), which as we noted earlier, is one type of NFR per the ISO/IEC/IEEE 29148 standard (ISO/IEC/IEEE, 2018). Some scholars treat the two as synonyms (e.g. Capilla et al., 2012).

A third issue, related to the issues above, is that much of the literature discusses NFRs as if they are similar. That is to say, while most scholars would readily acknowledge that, say, performance and maintainability are NFRs of a very different nature, most studies continue to discuss them as if they can be managed in similar ways. For example, studies discuss "the state-of-the-practice of NFR management in industry" (Borg et al., 2003), how "architects deal with NFRs in practice" (Ameller et al., 2012a), or how practitioners "tailor agile methods to handle" NFRs (Rahy and Bass, 2022).

While these challenges exist, we chose to retain the use of the term non-functional requirement in this study. First, documents such as the ISO/IEC/IEEE 29148 standard consider quality requirements a subset of NFRs, and so the latter term is more inclusive. Second, while we acknowledge that determining whether a given requirement is functional or non-functional can be ambiguous, we would argue that the intuitive meaning of "non-functional" requirement would be sufficient to discuss this topic with practitioners. To ensure the term was clear, we clarified the meaning of several terms, including 'NFR' for all interactions during this study.

NFRs are crosscutting, which means that, unlike functional requirements, they cannot easily be traced to a specific, isolated fragment of code (Bellomo et al., 2014; Glinz, 2007), nor can they easily be decomposed into code contributions within a single release cadence (Bellomo et al., 2014). Instead, NFRs are closely related to a system's architecture (Garlan et al., 1995; Harrison and Avgeriou, 2007). This challenge had already been observed for agile methods (Behutiye et al., 2020; Nasir et al., 2023; Rahy and Bass, 2022). As many software organizations abandon agile methods in favour of CSE approaches (Bosch, 2014; Fitzgerald and Stol, 2017; Olsson and Bosch, 2014), it is becoming increasingly challenging to maintain NFRs. Whereas agile methods deliver clearly distinct releases with a regular cadence, CSE entails a continuous delivery process, which means that new functionality is delivered much faster. As developers focus on

new functionality, an understanding of how NFRs are implemented and achieved may gradually and subtly decay.

Further, satisfying NFRs can be problematic because they are frequently ambiguous, not well documented, shared in an informal way, and poorly understood (Ameller et al., 2012a,b; Chung et al., 2000). NFRs are also inherently difficult to verify or validate (Borg et al., 2003; Jiang and Adams, 2015). The NaPiRE project ("Naming the Pain in Requirements Engineering") (Méndez Fernández, 2018; Wagner et al., 2019) found that "unclear / unmeasurable" NFRs were one of the most pressing problems respondents had in their small organizations (Méndez Fernández et al., 2017). Behutiye et al. (2020)'s systematic mapping study reported a wide range of challenges, including time constraints due to short iteration cycles. This challenge in particular is now exacerbated in CSE, which emphasizes even faster cycles than agile methods, and where requirements may be informally captured "just-in-time" (Ernst and Murphy, 2012).

Given the increasing popularity of CSE, there is a surprising lack of research on managing NFRs in such contexts and very little research on how NFRs are managed in agile contexts, with some notable exceptions. Karhapää et al. (2021) identified proactive, reactive, and interactive strategies to manage NFRs (using the term 'quality requirements') in agile software development, as well as no fewer than 40 challenges in so doing. Rahy and Bass (2022)'s multiple case study showed how companies using Scrum adopted different approaches to manage NFRs. For example, companies may treat NFRs as regular artifacts as part of the agile process, or companies may revert to conventional, plan-based practices to handle NFRs. Nasir et al. (2023) identified five documentation practices for NFRs. In previous work, we observed a series of practices that small companies have used to manage NFRs in CSE contexts, as well as associated challenges (Werner et al., 2022). In this article, we revisit this dataset to focus specifically on the shared understanding of NFRs in CSE contexts, rather than NFRs themselves. We discuss the importance of shared understanding next.

2.3 Shared Understanding of Non-Functional Requirements

Shared understanding refers to a form of communal knowledge or belief (Glinz and Fricker, 2015), "cognitive consensus" (Kleinsmann et al., 2010) among members of a group, and "mutual knowledge, mutual beliefs, and mutual assumptions" (Clark and Brennan, 1991). Shared understanding is a critical factor when working in a collaborative environment, as different people may have diverging understandings that are not mutually shared (Bittner and Leimeister, 2014). Glinz and Fricker (2015) describe shared understanding along two dimensions: true vs. false shared understanding, and implicit vs. explicit shared understanding. True shared understanding is when different people maintain an equivalent, common perception, whereas false shared understanding implies that people within a given group hold different assumptions, which can lead to considerable problems. The distinction between 'explicit' and 'implicit' is whether the shared understanding is specified and recorded in some form, or simply unspecified knowledge existing as an idea.

It is worth distinguishing shared understanding from two related (and possibly overlapping) but distinct concepts: tacit knowledge and domain knowledge. Tacit knowledge refers to "uncodifiable knowledge," or knowledge that is "difficult to articulate and dependent on skill and know-how acquired through experience" (Gacitua et al., 2009).

Tacit knowledge, in the context of requirements engineering, tends to refer to knowledge that might be known by system users, but not by its developers (Niknafs and Berry, 2017; Sporsem et al., 2023), though some scholars used the term to refer to expert knowledge among software development teams (Ryan and O'Connor, 2009, 2013). Domain knowledge refers to the "specialized understanding" of a given field in which a system operates (Araújo et al., 2025); this includes key concepts and terminology. Domain knowledge is important for software professionals to allow them to communicate effectively with other stakeholders in the development process (Ryan and O'Connor, 2013). Shared understanding, as we use the term in this study, refers to a certain type of knowledge that is shared among developers of a given system under development. The potential 'gap' in knowledge is not between users and developers, but among software professionals involved in the production of a system. In the context of this study, we focus on the mutual understanding, the "cognitive consensus," and shared interpretation of the NFRs of a system, among developers and key stakeholders involved in its production. In practical terms, shared understanding of an NFR means that there at least two individuals who have common and equivalent knowledge concerning a given NFR. Ideally, all relevant stakeholders have a shared understanding. Shared understanding is not a binary concept, nor is it directly measurable; rather, shared understanding of an NFR is a latent concept whose presence can only be inferred through some indirect, empirical indicators.

A lack of shared understanding means that people do not share common knowledge or beliefs. Developing shared understanding within software teams is a major challenge; the development of high-quality software and managing critical requirements pose huge demands on the attention of teams (Corvera Charaf et al., 2013; Hoffmann et al., 2013). A key goal of RE is to create shared understanding between development teams and other stakeholders (Fricker et al., 2015). However, the practice of creating and maintaining shared understanding is not well established (Schön et al., 2017). Misunderstanding of requirements early on in the development process can lead to substantial rework (Bjarnason et al., 2011). Unfortunately, shared understanding is often passive, informal, and unstructured (Glinz and Fricker, 2015), which negatively affects software quality and necessitates rework (Damian and Chisan, 2006). Given the cross-cutting nature of NFRs, the scope of rework may be considerable.

2.4 Summary of Current Understanding and Open Questions

There is little insight into how companies detect, build, and maintain a shared understanding of NFRs in CSE contexts. Prior work on shared understanding in general (notably Glinz and Fricker (2015)'s taxonomy) has suggested practices that companies may employ to detect such a lack of shared understanding. This line of work assumes that companies would *proactively* use such practices to detect a lack of shared understanding. Whether companies actually do this remains an open question. How and when companies employ such practices is also not clear. Further, while practices have been offered to build shared understanding (Glinz and Fricker, 2015), whether, when, and how companies implement such practices remains unclear.

The CSE literature suggests opportunities for fast feedback and learning (Fitzgerald and Stol, 2017; Johanssen et al., 2019; Klotins et al., 2023; Tkalich et al., 2025). Rooted in this is an assumption that such faster feedback and learning helps software developers to build and maintain a shared understanding of NFRs. Whether or not this is true

remains an open question. Finally, prior literature focused on managing NFRs does not acknowledge the inevitable 'decay' of shared understanding over time. Shared understanding is not something that can be built once and then assumed to remain intact. Rather, the continuous emergence of disturbances and perturbations within organizations, such as staff turnover, is likely to change the extent of shared understanding that people actually have.

To address these questions, we leverage insights grounded in fieldwork previously reported (Okpara et al., 2022, 2023; Werner et al., 2020, 2022) (see Table 1) to develop a theory that describes and explains how small software companies detect a lack of shared understanding of NFRs, and how they build such shared understanding in CSE contexts. In previous work, we developed a number of insights regarding the management of shared understanding of NFRs. For example, we identified three factors that contribute to a lack of shared understanding of NFRs, as well as practices to build shared understanding, and several associated challenges (Okpara et al., 2022, 2023; Werner et al., 2020, 2022).[3] Earlier work also proposed the concept of a 'trigger' to identify a need to build a shared understanding of a certain NFR (Werner et al., 2020). However, what is notably missing is a holistic and overarching explanation that describes and explains the intricate relationships between these practices, challenges, and triggers. The present research revisits this dataset and the various categories that we previously identified to engage in theory development as a research strategy (Stol and Fitzgerald, 2018). In doing so, we develop a process theory that captures the dynamics of how the various categories previously identified relate to one another. In this article, we provide a detailed description of this theory.

3 Methods

3.1 Research Strategy

This article concludes a research program that has focused on the shared understanding of NFRs in continuous software engineering contexts in small software companies. Drawing on these empirical findings (Okpara et al., 2022, 2023; Werner et al., 2020, 2022), in this article we set out to develop a theory that parsimoniously captures our understanding of this phenomenon. Organizational theorist John Miner described theory as follows (Miner, 1980, p. 3):

> "Theory is a patterning of logical constructs, or interrelated symbolic concepts, into which the known facts regarding a phenomenon, or theoretical domain, may be fitted. A theory is a generalization (applicable within stated boundaries) that specifies relationships between factors. Thus, it is an attempt to make sense out of observations that do not contain any inherent and obvious logic."

Whereas our preliminary studies made several observations and insights, what has been lacking is an apparent logic that explains how these observations tie together. What follows in the remainder of this article is (a) a description of our process of theory development, and (b) the outcome of that process. We adopted an inductive

[3] The various papers have been co-authored by different sets of investigators, but has been primarily led by the lead investigators Werner and Okpara. In this article, we draw on the data collected from the fieldwork conducted by Werner and Okpara, which is why we refer to ourselves as 'we.'

Table 1 Overview of our previously published field studies

Study	Methods and data	Contribution
Werner et al. (2020)	3 case studies (Alpha, Beta, Gamma); interviews and informal conversations with 37 people, artifact analysis, focus groups	Develops an under st anding of 3 factors that contribute to a lack of shared understanding of NFRs: fast pace of change (related to CSE), lack of domain knowledge, inadequate communication. Identifies practices to alleviate these factors (shared development standards; documentation and communication); differentiation of 'essential' and 'accidental' lack of shared understanding of NFRs. Four triggers that led organizations to build shared understanding of NFRs: regulatory requirements; accumulating technical debt; needs of important customers; and disruption of service.
Werner et al. (2022)	3 case studies (Alpha, Beta, Gamma); interviews and informal conversations as in Werner et al. (2020)	A set of 4 practices to handle NFRs in CSE contexts: put a number on the NFR; let someone else manage the NFR (write your own tool to check the NFR; put the NFR in source control), and 3 associated challenges (not all NFRs are easy to automate; functional requirements get prioritized over NFRs; lack of shared understanding of an NFR). Trade-offs associated with these practices in CSE contexts.
Okpara et al. (2022), extended as Okpara et al. (2023)	1 case study (Delta); participant observation; 21 interviews	5 practices for building a shared understanding of NFRs (validating NFRs through feedback; deepening the understanding of NFRs through experience; wireframing interface designs; discussing problems and solutions; asking questions), and proactive vs. reactive characterization of these; 4 limitations to building shared understanding of NFRs (gaps in communication; limited understanding of customer context; individual differences; unspecified NFRs).

theory development approach (Eisenhardt, 1989; Glaser and Strauss, 1967; Miles and Huberman, 1994; Ralph, 2019). While several approaches to theory development exist, ours aligns closely to Eisenhardt (1989)'s, as we elaborate in the remainder of this section (see Table 2).

3.2 Cases and Data

For the present article, we drew on a dataset compiled through four case studies, which are named Alpha, Beta, Gamma, and Delta, and which we previously reported on (Okpara et al., 2023; Werner et al., 2020). We engaged in additional rounds of analyses drawing on theory development guidance, which involved a process of 'fitting pieces together,' as one would do in a jigsaw puzzle (Stol and Fitzgerald, 2018).

We selected the cases based on their relevance to the research, namely, small companies using CSE practices. Appendix A presents more details on the selection and background of the cases, as well as details on the procedures followed for conducting observations, interviews, artifact analysis, and focus groups. The first three case studies were conducted in 2019, and the fourth case study in 2021-2022. Overall, this extended time frame allowed time for analysis and reflection to develop new theoretical insights. This longitudinal approach enabled the development of a provisional theory that could

Table 2 Summary of theory development steps following Eisenhardt (1989)'s approach to theory building from case study research

Step	Description
Getting started	A rigorous literature review led to the definition of our research question, defined in Sec. 1: How do small software organizations maintain a shared understanding of non-functional requirements in fast-paced continuous software engineering contexts?
Selecting cases	Based on the research question, we selected small companies that used continuous software engineering practices. See Sec. A.1 for details.
Crafting instruments and protocols	Data collection methods and instruments were prepared, including interview guides. Sec. A. Further details are described in earlier publications (Okpara et al., 2022, 2023; Werner et al., 2020, 2022).
Entering the field	Researchers spent multiple days over a few months embedded in situ. Details are provided in Sec. A.2.1, including on observations, memoing, interviews, and informal conversations.
Analyzing the data	Data were analyzed following qualitative data analysis procedures, by multiple investigators. This included open coding (labeling of concepts, etc.; see Fig. 1), axial coding (labeling of relationships; see Table 3).
Shaping hypotheses	The concepts and relationships between these concepts that emerged during axial coding led to the development of a set of tentative propositions. Through a process of "verification of statements against data," "a constant interplay between proposing and checking" (Strauss and Corbin, 1990), these tentative propositions were sharpened and finalized.
Enfolding literature	Comparison of emergent theory to prior literature to identify similarities and contradictions (Eisenhardt, 1989). For each proposition, we studied relevant prior literature, and declared whether the proposition is contradictory, in agreement, or adding nuance. This also includes attempting to find explanations in light of any contradictions.
Reaching closure	Eisenhardt (1989) suggests a number between 4 and 10 cases; this theorizing study is based on secondary and supplementary data analysis of 4 cases. We judged that there was sufficient evidence or justification for each of the proposition based on these 4 cases. We concluded iteration between theory and data once we felt that the propositions sufficiently described the process that we sought to study.

be further evolved based on insights learned in the fourth case study. For example, based on the fourth case we distinguished between *reactively* and *proactively* building shared understanding—which we reflected in two separate propositions of the emergent theory.

We developed a set of propositions that together were combined into a dialectic process theory.[4] A dialectic process theory captures an "interplay between two or three activities, actors, concerns, forces, or organizational logics" (Ralph, 2019). Such a process tends to describe "conflicting entities and explain how these conflicts shape ongoing change" (Ralph, 2019). These conflicting forces include the fast pace of software development dictated by CSE approaches, which plays a role in the decay of shared understanding of NFRs, and the practices used by stakeholders to build that shared understanding.

[4] Many of the analysis procedures are part of, and popularized through the grounded theory method (Glaser and Strauss, 1967; Glaser, 2009; Miles and Huberman, 1994). However, while our study is an inductive one, it is not a grounded theory study, because data analysis did not start immediately, a key feature of grounded theory (Glaser, 2009; Stol et al., 2016). Further, the study was guided by a clearly defined research question that we identified through an extensive literature review; it did not 'emerge' from the data.

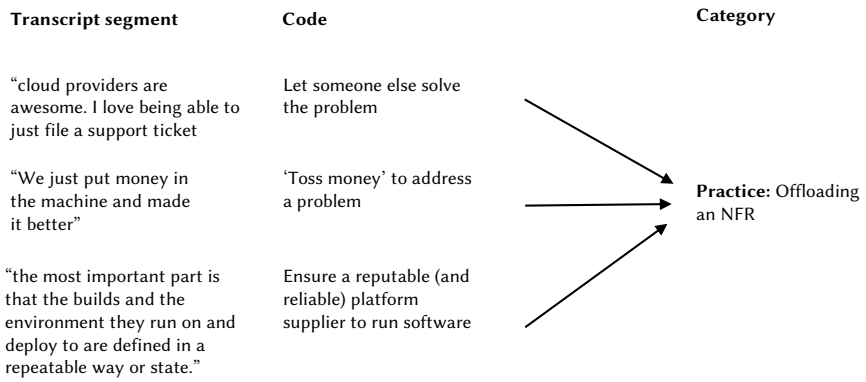

Fig. 1. Example of coding

Table 9 (see Appendix A) provides an overview of the various types of data that underpin the theory development. These data each served a role in developing an overall understanding of the context of the cases, practices, challenges, and the nature of rework that could be linked to a lack of shared understanding. The memos written during observations provided descriptions of the case companies, capturing their business domains, ways of working, practices and tools. We discussed these memos among the team to establish a shared foundation for further analysis. Data from the focus groups and interviews were analyzed using thematic analysis; we used inductive open coding to generate codes to label segments of the transcripts pertaining to specific themes (Corbin and Strauss, 1990; Cruzes and Dyba, 2011; Seaman, 1999). Initially, two investigators independently coded each dialogue segment of the same transcript before 'calibrating' with the other investigator. The calibration involved comparing codes and notes, and any discussion to resolve disagreements and consolidate codes, to come to a common understanding of the themes at hand. As part of these calibrations, we calculated Cohen's kappa coefficient. This initial coding phase continued until the kappa value stabilized above 0.6, which indicates substantial inter-rater reliability agreement (Landis and Koch, 1977). After this initial coding phase, each of the remaining transcripts was coded by a single investigator and reviewed by a different investigator. During the analysis we used the constant comparison method (Corbin and Strauss, 1990; Glaser and Strauss, 1967; Seaman, 1999); events, incidents and examples were constantly compared, asking, "of what category is this an instance?" Through this process, new codes were developed, and on occasion, they were merged, for example, when two different but similar codes had been used to refer to the same category of event or incident; this merging happened after discussions among investigators (Glaser and Strauss, 1967; Seaman, 1999). Through this process of coding transcripts, we captured a variety of themes, including incidents, ways of working, perceptions, and challenges. During the coding process, we wrote memos to capture our observations and reflections. An example of the coding process is presented in Figure 1. The figure shows that transcript segments were labelled with short phrases as codes. These codes helped to capture the sentiment or essence of a transcript segment. Codes were then sorted and organized into categories; the figure shows that the three interview segments all described a practice of what we later labelled 'offloading an NFR.'

Table 3 Identifying relationships among concepts: antecedents and consequences

Transcript segment	Memo	Theme
"Availability is actually an interesting one, do you spin up Amazon services in Virginia, or do you avoid that because that's their biggest region and so things sometimes go wrong there." (Alpha)	Manager talks about how they become reliant on a cloud provider to provide an NFR, but they are also at the mercy of the cloud provider, resulting in a loss of control over that NFR.	Offloading an NFR is *susceptible* to loss of control
"It's a scalability issue, but it's not prioritized, so we need to build more domain knowledge so it doesn't fall by the wayside." (Beta)	Developer and manager indicate that scalability was an issue, but was not a high enough priority due to a lack of shared understanding.	Deprioritization *results in* lack of shared understanding
"The developers working on the code didn't know the impact causing an outage, this is one of those things that people don't pay too much attention to until they need to." (Gamma)	A lack of shared understanding of transparency resulted in a loss of service for a customer.	Lack of shared understanding *manifests as* a service disruption

We used member checking as a practice to establish the validity of our analysis. We used a short online survey to check whether our intermediate analysis findings resonated with the informants which presents intermediate findings through an online survey. Based on the feedback, we abandoned some themes that did not resonate with participants and made further amendments as our analysis continued.

3.3 Shaping Propositions

As our analysis continued towards the development of propositions, we also identified links between different categories. We used practices akin to what Strauss and Corbin (1990, p. 96) labelled axial coding; that is, "making connections between categories." This process involves asking questions about the phenomenon that we observed (e.g. "what is the action/interaction all about?" (Strauss and Corbin, 1990, p. 100-107)), causal conditions that led to the phenomenon, and consequences or outcomes of the phenomenon. During this process, we captured our observations and deliberations in short memos. Table 3 shows an excerpt of this process. For example, we identified 'loss of control' as a potential consequence of the practice of 'offloading an NFR'; a lack of shared understanding as a consequence of de-prioritizing NFRs, and a service disruption as a manifestation of a lack of shared understanding. Part of these analysis procedures was the sketching of our emergent propositions and identifying links among categories (Eisenhardt, 1989; Miles and Huberman, 1994). Ultimately, the development of these propositions is subjective, insofar that different investigators are likely to come up with an alternative interpretation, because each investigator has a certain background and experience which naturally influences their interpretation and ability to identify patterns. We discuss our positionality below, and discuss the inherent subjectivity of the analysis process as a threat to validity.

After analyzing the data from cases Alpha, Beta, and Gamma, we established initial propositions that together formed an emergent process theory. This emergent theory was then further refined as we collected and analyzed data from the fourth case study at Delta.

Table 4 Example of refining propositions

Proposition	Earlier version	Final version
Proposition 2	A trigger prompts an organization to implement a practice to build shared understanding of a non-functional requirement.	Management practices are primarily used to reactively build a shared understanding of non-functional requirements, in response to triggers.
Proposition 3	An organization may also implement a practice unprompted.	Communication practices are primarily used to proactively build a shared understanding of non-functional requirements.

At this point, we engaged in what Strauss and Corbin (1990, p. 108-111) described as "verification of statements against data" and "a constant interplay between proposing and checking." This entailed an iterative process of adding and refining statements to the emergent theory, while ensuring that prior statements remained valid as new insights were generated. The fourth case study, therefore, served the purpose not only of verification of the initial set of propositions but also to add further nuance to the emergent theory. For example, a key contribution of the case study at Delta was our observation that certain practices were used proactively to build a shared understanding of NFRs (see Proposition 3, discussed later), something we had not observed in the first three case studies. To illustrate the refinement process: Table 4 shows an early version of Proposition 2, and of Proposition 3, which originated in insights gained from the fourth case study. Upon closer inspection of the practices that we identified, we observed that the practices that were use in response to a trigger appeared categorically different in nature than those that were used proactively. The former allowed developers to manage, monitor, or quantify an NFR, and so we called these 'management' practices. The latter, on the other hand, appeared to focus on information exchange, and so we called these communication practices. This distinction then led to the refinement of these propositions.

3.4 Positionality of the Investigators

We now briefly discuss the positionality of the investigators. The lead author has worked in the software industry for almost 20 years. He is currently a Director of Engineering, and has previously worked as an operating system specialist and software developer. He holds a PhD in Computer Science. His experience of having worked in both large and very small companies has influenced his vision on software engineering as a practice, but also as a field of research. The second author has worked in the IT industry for 10 years in varying roles. She holds a Master's degree in Computer Science. The third and fourth authors are professors of software engineering, and both have primarily an academic focus on research. Both have extensive experience in conducting fieldwork, qualitative data analysis, and theorizing (cf. (Damian and Zowghi, 2002; Stol et al., 2014)).

4 The Ebb and Flow of Shared Understanding of Non-Functional Requirements in Small Organizations using Continuous Software Engineering

We now present the results of our theory development, which comprises six propositions. For each proposition, we juxtapose suggestions, findings, or assumptions from prior literature, with the findings from our fieldwork. We present a visual presentation of the theory in Figure 2.

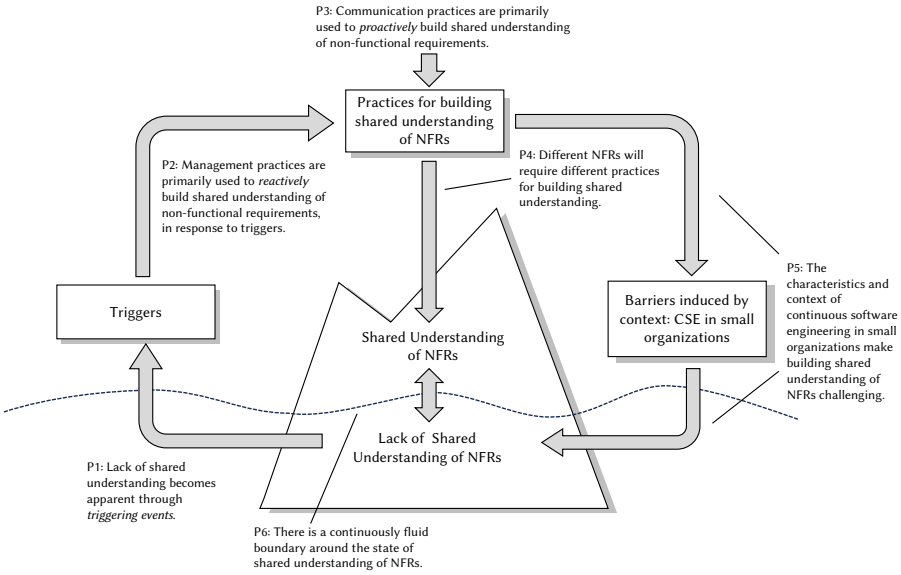

Fig. 2. The Iceberg theory of managing shared understanding of non-functional requirements in continuous software engineering contexts

4.1 Detecting Lack of Shared Understanding of Non-Functional Requirements

How might a lack of shared understanding manifest or be detected? Glinz and Fricker (2015)'s taxonomy suggests that shared understanding is either true or false: either members of a team share an understanding (i.e., a true shared understanding), or they believe they have a shared understanding, but in actual fact, they do not (i.e., a false shared understanding). To determine whether a group of people have a shared understanding, Glinz and Fricker (2015) identified and discussed 11 practices. For example, to create and play out scenarios to evaluate whether everybody understands how a system will work in these given scenarios. There are two potential issues with this set of practices. First, they cannot *necessarily* identify a specific lack of shared understanding of a given topic or part of a system. Second, and perhaps more importantly, most practices appear to assume that a team *proactively* performs these practices to establish whether there is a shared understanding.

The findings of our field study suggest that a lack of shared understanding is not proactively assessed, but instead is revealed through emergent critical events. We call these 'triggering events' (or 'triggers' for short (Werner et al., 2020)), which make team members aware that there is a lack of shared understanding of an NFR. Or, formally stated:

Proposition 1: *A lack of shared understanding of non-functional requirements becomes apparent through triggering events.*

Table 5 presents data that grounds this proposition. In all cases, we found that there are triggers that led to a team's realization that they lacked a shared understanding of specific NFRs. Our analysis revealed five categories of triggers, though it is likely that these are not exhaustive. We elaborate on these categories below.[5]

Table 5 Categories of triggering events and selected examples

Category	Example	Description	Company
New requirements	"So they wanted to extend it and could not figure out how without doing all this rework to start with."	Lack of knowledge was revealed when new requirements are to be implemented.	Alpha
	"You might get halfway through implementation and realize there are a whole bunch of questions that you don't know the answers to. And those questions should have been answered but weren't, either because no one understood at the time or because you've discovered new requirements."	Questions arose during implementation, identifying new requirements.	Delta
	"something out of our control changed, or other things have changed since that was put into place due to privacy regulation acts"	Privacy regulation as a source of new requirements.	Gamma
Customer demands	"[this issue was a] big problem for one user, one very big user, because for [redacted customer's name] everything that company says is critical."	An NFR-related issue became a high priority rework task when an important customer reported it.	Beta
	"if your requirement is still the same and your system is still working as it was, but it's the actual environment that has changed."	The customer's environment differs from the development environment, highlighting an issue e.g. portability.	Gamma
Technical debt	"any time you needed a place to put something we used to toss them on that big stack of unorganized classes until it became a giant object."	Lack of shared understanding regarding the trade-off between taking on technical debt and maintainability. A lack of shared understanding led to continued behaviour, ultimately requiring very significant rework.	Beta
	"two people [developing] independently, they talk to each other well, but you're still going to approach problems differently and hit different areas of the system and not know everything that's going on."	Misunderstandings become clear as developers discuss how to refactor code to address technical debt.	Beta

[5] The set of triggers presented here is a revised set of categories after we initially reported these in Werner et al. (2020).

Table 5 Categories of triggering events and selected examples (continued)

Category	Example	Description	Company
Scope creep	"start an app early on for North American companies requiring one [format]. As your app becomes more popular you start getting requests for new countries that use [other] formats"	Lack of shared understanding of configurability.	Gamma
	"they had made the assumption that they would be able to go in and update these prints after launch. And we had made the interpretation that everything was just sort of hard-coded, write once and forget it. And so, in the end, it was something that we actually had to push back on the client and say that, making that something that's editable, would significantly increase the scope."	Need for major rework to make a system that is more flexible that had not been foreseen.	Delta
Service disruption	"The rework on it was not only to refactor it [...] but essentially to handle the type of structure used [in the application] that caused problems throughout the app."	An NFR-related issue caused considerable rework due to what turned out to be a lack of shared understanding.	Alpha

New requirements The first category of triggers is the emergence of new requirements: during design and implementation discussions, developers may realize that parts of the design were not sufficiently specified. For example, at Alpha, some functionality had to be extended, and developers were unclear as to how to achieve this without considerable rework. The discussion among developers led to the realization that they lacked a shared understanding. Many NFRs are driven by a specific business domain; for example, performance was key to three of the four organizations we studied (see Table 9). Other NFRs may emerge due to regulation; for example, the GDPR mentioned earlier, came into effect in 2016, and requires that organizations collecting or processing data in the EU must comply with specific privacy regulations. This, too, imposes new requirements which may lead to the realization that teams lack a shared understanding of how to implement such privacy requirements.

Customer demands We identified a second category of triggers that we labelled 'customer demands.' On occasion, customers would offer feedback, reporting problems with NFRs such as performance. This would then lead to an investigation of the issue, and a realization that the team did not share an understanding of a specific NFR. For example, at Beta, a problem emerged due to poor performance in the system deployed to one major customer. Escalation of the issue would depend on the scale of the problem, but also the customers affected. Or, as a developer at Alpha mentioned: "if a lot of customers had this setup, then [the priority of the issue] would have been high."

Technical debt A third category of triggers is issues arising due to technical debt. For example, a lack of shared understanding of maintainability for a product at Beta caused issues because developers writing new functionality would simply "toss them on that big stack of unorganized classes until it became a giant object." While the initial developer, who placed new functionality in a 'utility' component or class without proper integration into a coherent architecture, may have made a conscious decision to trade-off technical debt against maintainability, this action had consequences. This initial action

served as a precedent for other developers to continue this practice without consciously acknowledging the trade-off that had been made earlier. Ultimately, this 'snowballing' of continuing to follow initial precedent led to problems affecting the whole app, meaning that considerable rework was necessary. Developers would try to resolve defects, but in actual fact, they were exacerbating the overall technical debt. This would continue until such time that a decision was made to address the underlying issue and undertake a major refactoring. To perform the refactoring, however, developers now had to come to a shared understanding of how to improve the maintainability, and redesign parts of the application.

Scope creep A fourth category was labelled scope creep, which has been defined as "the growth of the scope of a project's requirements beyond those originally intended" (Williams and Williams, 2009). In this context, the growth of scope related to NFRs arose due to an initial lack of understanding of the requirements. For example, at Gamma, an initial version of the system under development focused exclusively on the North American market, with its specific standards, notations, and customs. Only when the team was instructed to deploy the system to other countries (with different standards and notations), was a lack of shared understanding of the need for configurability detected.

Service disruption Finally, a fifth category of triggers is service disruption; triggers in this category refer to critical events that interrupt the software from working correctly, due to a lack of shared understanding of NFRs. For example, a lack of shared understanding of scalability turned out to be a major problem at Alpha, leading to a service disruption. A developer at Alpha explained that, rather than refactoring the code, to prevent future breakdowns, it had to be redesigned to "handle the type of structure used that caused problems throughout the app."

Why does a lack of shared understanding get detected through triggering events, rather than proactive assessment by teams? One reason may be that many practices that can help to assess shared understanding are based on explicit documentation or artifacts (Glinz and Fricker, 2015) and that assessment activities such as creating and playing scenarios, model checking, and comparing to reference systems (Glinz and Fricker, 2015), all require teams to proactively allocate time to perform these activities and capture the information that is gathered. In small companies, in particular, such resources may not be available.

A second reason may be the pressure to keep delivering new versions of the software in CSE settings. Many of the practices in CSE can be linked to 'Lean Thinking' (Fitzgerald and Stol, 2017), emphasizing minimizing waste, which means removing unnecessary activities and not creating unnecessary artifacts. We found no evidence in our fieldwork of any proactive attempt to assess or identify a lack of shared understanding of NFRs. Further, our focus on small organizations, all of which started as startups about 10 years before we conducted our fieldwork, underscores this. The Lean Startup movement, itself influenced by principles of Lean Thinking (Ries, 2011), emphasizes reducing the time between a feature request and 'cash' (Fitzgerald and Stol, 2017).

Finally, practices to proactively assess a lack of shared understanding do not appear to focus on any specific NFR. All data collected in this study suggest that a lack of shared understanding is only realized when problems with a specific NFR occur. For example, reliability turned out to be poor for Alpha when their service was disrupted. Performance turned out to be poor for Beta when an important customer reported problems that

required urgent attention. While practices for proactively assessing a lack of shared understanding could work in general, they lack focus on specific NFRs.

4.2 Building Shared Understanding of Non-Functional Requirements

How might developers build a shared understanding of NFRs in CSE settings? Prior literature suggests a number of enabling practices to build shared understanding (Glinz and Fricker, 2015). Glinz and Fricker argued that these practices can "constructively [lower] the probability of undetected misunderstandings." Practices as reported in prior literature can be effective in sharing knowledge and building a shared understanding of requirements, including NFRs. However, our study offers a more nuanced account of how organizations manage a lack of shared understanding of NFRs. We found that organizations do not necessarily proactively invest in building a shared understanding of NFRs, but that they do so in response to certain critical triggering events, discussed earlier. Drawing on insights from the general requirements engineering and knowledge-sharing literature, Glinz and Fricker (2015) identified several practices for enabling and building shared understanding in general. Enabling practices include activities such as team building, collaborative learning, negotiation, and prioritization. Building practices include activities such as modelling, creating glossaries and ontologies, prototyping, workshops and handshaking (Glinz and Fricker, 2015).

While NFRs are a particular type of requirement, the literature has been quiet on how a shared understanding of NFRs, specifically, is developed. Much of the state of the art, as compiled by Glinz and Fricker, is presented as a set of unproblematic practices that can be performed without any issue. The findings of our study offer new insights into this. First, we distinguish between what we label 'management' practices and 'communication' practices. Management practices include activities that allow the measurement and monitoring of NFRs and are primarily invoked in response to triggers. In contrast, communication practices include activities that leverage interaction between developers and other stakeholders to clarify NFRs and are primarily used to build shared understanding proactively. Thus, more formally, we posit:

Proposition 2: *Management practices are primarily used to reactively build a shared understanding of non-functional requirements, in response to triggers.*

Proposition 3: *Communication practices are primarily used to proactively build a shared understanding of non-functional requirements.* Further, we also found that different practices are likely necessary for building a shared understanding of different NFRs. Hence, we add:

Proposition 4: *Different non-functional requirements require different practices for building shared understanding.*

Table 6 presents data from the fieldwork, organized by the practices that we distilled.[6] We note the specific NFR that was the topic of discussion or indicate 'all' if the description is generally applicable. Each practice is labelled as either a management or communication

[6] Table 6 presents a revised set of practices that were previously reported in Werner et al. (2020, 2022) and Okpara et al. (2022, 2023).

practice. We discuss evidence for Propositions 2, 3, and 4 in Sections 4.2.1, 4.2.2, and 4.2.3, respectively.

Table 6 Evidence for Propositions 2, 3, and 4: Practices for building Shared Understanding of NFRs

Practice	Example	Description	Company / NFR	Practice type
Quantifying an NFR	"a certain amount of monitoring set up [...] You're also defining which alarms are set. Therefore, if requests [drop] below [redacted] milliseconds and that's what you want to hold it to. Then that would be codified in the alarm"	Monitoring infrastructure and continuous monitoring help to establish baselines and thresholds that assume 'meaning' among developers, who will establish a shared understanding of how to interpret these numbers and what actions might have to be taken.	Alpha; scalability	Management
	"that 90% of our customers have less than [redacted] items [...] They'll [say] we know that each [order] requires [redacted] page loads"	Quantification allows establishing trends across different customers, which means that variables such as 'page loads' now convey meaning among team members.	Beta; performance	
	[tracked through a] "[caching ratio] that we have from [redacted] our caching provider goes [on a dashboard] because that's usually a pretty good indication that something has gone wrong. It also drastically affects performance."	The caching ratio in the application is an indicator of performance; quantifying and displaying this number will help establish a common understanding among the team of when something might be going wrong.	Gamma; performance	
Offloading an NFR	"the most important part is that the builds and the environment they run on and deploy to are defined in a repeatable way or state."	To offload an NFR, To offload an NFR, appropriate configuration scripts must be written; the process of developing such scripts requires an agreement on that NFR at that point, hence it temporarily contributes to building shared understanding of that NFR.	Alpha; reproducibility	Management
	"cloud providers are awesome. I love being able to just file a support ticket"	Offloading NFRs such as reliability requires some shared understanding to design the software to work with the cloud provider in the first place.	Beta; reliability	

Table 6 Evidence for Propositions 2, 3, and 4 (continued)

Practice	Example	Description	Company / NFR	Practice type
	"We just put money in the machine and made it better"	To increase performance, more compute resources can be acquired from a cloud provider, but the software must be designed to allow this scalability. To achieve this, a shared understanding of NFRs such as performance must be achieved first.	Gamma; performance	
Develop custom tools and standards	"If you want to change our usability parameters or whatever we change it in the source code and then we can test it and verify that it still meets our needs"	Development standards and tools may include templated solutions, or simply the codification of key variables of interest as parameters; the code and interpretation of these parameters are visible to the team to help build shared understanding.	Beta; usability	Management
	"so we used to have [redacted commercial offering] dashboards out there ... [but that] didn't give us any application-specific information"	Developing a custom support tool focuses on domain-specific metrics as opposed to basic metrics (CPU, memory, etc). These domain-specific metrics are then published in source code and dashboards to facilitate building a shared understanding.	Gamma; performance	
	"I introduced a number of standards and built some packages to try and enforce those standards. So there's the standard for how we, on the server side, get configuration from the environment"	Developing custom libraries or standards implies that all team members must learn these as agreed upon 'protocols', implying that a shared understanding is built.	Delta; maintainability	
Capturing an NFR in version control	"with respect to codifying something vs documenting it: it's not precise enough it's written in English. It's open to interpretation"	Capturing NFRs in natural language lacks precision which may cause decay of shared understanding. Establishing exact threshold or cut-off values allows for an unambiguous definition of NFRs, contributing to a shared understanding.	Gamma; configurability	Management

Table 6 Evidence for Propositions 2, 3, and 4 (continued)

Practice	Example	Description	Company / NFR	Practice type
Solicit feedback	"there are things like code review, review and feedback from the managers that sort of catches the NFRs"	Feedback comes from different sources including code review, colleagues and managers. Such feedback may incorporate clarification contributing to a shared understanding among team members.	Delta; maintainability	Communication
	"our [deep-dive] during the development process and the project engineering role really helped with that. Because when the shaping is happening, we're starting to think about those non-functional requirements: how it might work for other use cases, how something should be developed from a quality perspective and what tools we are using"	Feedback during a deep-dive meeting helps to clarify NFRs, thereby contributing to a shared understanding of these NFRs.	Delta; usability	
Leverage past experience	"I think the biggest tool that has helped me learn NFRs has really just been reviewing previous work, specs, and integrations."	Discussing how NFRs were previously implemented helps build a shared understanding among the team.	Delta; all	Communication
	"there's a process that isn't formally presented, it's picked up over time through training. I think that the majority of NFRs have been picked up that way"	Training over time helps build shared understanding of NFRs.	Delta; all	
Wireframing	"something like Figma or Figjam to quickly visualize what we're talking about."	Visualizing as a form of prototyping helps to build a shared understanding of a system's usability.	Delta; usability	Communication
	"that's where things like 'shaping,' wire-frames, starting to try to think through that user experience really helps. Design drawings, I find are really helpful. Early mock-ups help people because I find that a lot of the team does well with actually experiencing something."	Creating mock-ups helps a team of developers to "experience" a system as if they were users, thereby creating a shared undersatnding of usability.	Delta; usability	

Table 6 Evidence for Propositions 2, 3, and 4 (continued)

Practice	Example	Description	Company / NFR	Practice type
Knowledge sharing activities	"it's been helpful for talking about how we build things and non-functional requirements in specific areas of our jobs"	Facilitating organized opportunities (e.g. 'guild' meetings) to share knowledge on for example implementation, helps to build shared understanding of NFRs.	Delta; all	Communication
	"Great user experience is one of those things that's difficult. When it comes to user experience, we've centralized the people who make user experience decisions into a group and then we have standards in that group"	Bringing together all people involved in system user experience helps to share and codify relevant knowledge.	Delta; usability	
Asking questions	"you want to ask a lot of questions, you want to poke a lot of holes and things and, really go to the very edge of, you know, when you're sketching it out."	While knowledge sharing activities are useful to develop a shared understanding of 'known' knowledge, what others have called 'unknown unknowns' are harder to share. Asking questions is a complementary practice to probe and explore some knowledge that might otherwise not be shared, because it was unknown, unthought of, or tacit in nature.	Delta; all	Communication

4.2.1 Management Practices

The first set of practices is classified as management practices, so called because they allow developers or teams to manage, monitor, quantify, and make an NFR explicit.

Quantifying an NFR A first management practice is quantifying an NFR, which is a management practice because the ability to do so allows an organization to assess, validate, and monitor the NFR over time. This, in turn, creates a feedback loop in the form of information about how the system performs in terms of NFRs, which feeds back into the development cycle. Quantifying NFRs as a practice is consistent with Wagner et al. (2019)'s finding that approximately half of the respondents to their large-scale survey used "quantified textual documentation for non-functional requirements." At Beta, the CSE approach enabled a fast feedback loop, which allowed a quick detection of defects causing system crashes, as one developer described: "I think quick feedback is the core of DevOps so that we will see what broke [and] it reduces risk because things are integrated more frequently."

The feedback loop is most useful if it is *fast*, and many organizations strive to reduce the time required for a feedback loop to complete. Quantifying an NFR is typically

done with dashboards that help with the proliferation of shared understanding of those monitored NFRs. A dashboard may be shared across an entire organization, stretching beyond the barriers of the development team, as all units of an organization, from finance to marketing, embrace the 'continuous *' approach (Fitzgerald and Stol, 2017).

Many organizations use metrics to monitor common NFRs, such as availability, performance, scalability, availability, and reliability (Werner et al., 2022). For Gamma, performance is a vital NFR and is tracked through a caching ratio retrieved from a caching provider, and captured on a dashboard, said one developer:

> "[the caching ratio] that we have from [...] our caching provider goes [on a dashboard] because that's usually a pretty good indication that something has gone wrong. It also drastically affects performance."

We found a similar setup at Alpha, as one manager explained how warnings and alarms are based on thresholds: "if requests [drop] below [a certain number of] milliseconds and that's what you want to hold it to. Then that would be codified in the alarm." At Delta, performance is primarily monitored through Datadog (Datadog, 2025), a commercial cloud-based monitoring offering; metrics such as error rates, average response times for API calls, and number of application instances are indicative measures of the state of their systems and applications.

Some NFRs are easier to quantify than others. As Wagner et al. (2019) observed, performance is more easily quantified than, for example, maintainability. One way for an organization to continuously validate usability is through iterative end-user testing or behavioural testing with stakeholders, by leveraging tools that support behavior-driven methodology. A manager at Beta explained how they use this approach to determine, for example, that 90% of their customers have less than a certain number of products in their basket, and that each order requires a certain number of page loads, in turn affecting usability. The page load count is, therefore, a variable that can be used in the design and development of a product basket, and to assess how usability might be affected. At Gamma, we found metrics to play a key role in managing usability across cross-functional teams, including development and product management, as one developer explained: "showing how many users are using a specific feature, where the feedback benefits developers but [also] our product team in their ability to make their decisions." Additionally, monitoring usability may be accomplished by embedding components in the user interface that report statistics back to the organization, enabling the organization to understand how their software is being used, what features are and are not being used, and how efficient their user interface is to use.

Offloading an NFR A second management practice is offloading an NFR, which we found to be the most common approach among the small companies that we studied. Small companies tend to be more resource-constrained than large organizations, which means that they may need to set stricter priorities in terms of the NFRs they focus on. Offloading NFRs that have a lower priority allows small companies to do so. Offloading an NFR allows an organization to effectively "ratchet" (Bellomo et al., 2014), as a manager at Gamma concisely described: "we just put money in the machine and made it better." Ratcheting allows an organization to consume more cloud provider resources or access locked features to handle a particular NFR bottleneck.

In practice, offloading an NFR can imply the use of configuration scripts, or Infrastructure as Code (IaC), to facilitate the bridge to cloud providers. This means that,

while the NFR itself, such as availability, can be 'secured' as the company now uses compute resources available with cloud providers, there is still a need to gain a good understanding of how the NFR can be achieved and implemented in terms of making appropriate configurations. It is this process that helps in building a shared understanding of the NFR.

Many NFRs, such as availability, scalability, and reliability, can be offloaded either entirely or partially, typically to a cloud provider. Cloud providers also offer support when issues arise. A developer at Beta described this succinctly: "cloud providers are awesome. I love being able to just file a support ticket." For example, a manager at Alpha explained the benefit of offloading to achieve scalability, without the need to invest in resources such as in-house hardware: "[your web application is] immediately spread across however many availability zones and nodes as you want."

Offloading NFRs is a viable tactic, particularly in CSE contexts. CSE emphasizes automation, including the development and deployment environments; such IaC increases configurability in general, and this, in turn, reinforces the benefits of CSE approaches (Humble and Farley, 2010). IaC has grown to capture more quality aspects of software, including build, staging, and deployment infrastructure, with very little input required from humans (Humble and Kim, 2018). A manager at Alpha described how IaC scripts capture the configuration: "For anything that I do, [infrastructure as code] ends up in Terraform as a form of documentation." Besides improving configurability, it also enables organizations to codify other NFRs captured as IaC. A developer at Beta described how the complexity of the system becomes more maintainable: "it automatically does it in the Docker compose files. So we've automated a lot of dependency updates and stuff like that."

Custom tools and standards. A third management practice is developing custom tools and standards. Off-the-shelf solutions may not satisfy an organization's specific needs and may need some heavy modifications, as one developer at Gamma pointed out: "so we used to have [tool name] dashboards out there ... [but that] didn't give us any application-specific information." We found that companies use custom tools, or customize existing tools, to enforce specific NFRs. For example, Beta and Gamma each codified certain usability parameters in their source code, leaving no room for misunderstandings. That way, the results of any changes can be observed after the committed changes have propagated through the CSE pipeline, as one developer at Beta described:

> "If you want to change our usability parameters or whatever, we change it in the source code and then we can test it and verify that it still meets our needs."

At Delta, developers defined and shared development standards within packages and custom tools to define how some NFRs are managed. A manager at Delta described this:

> "I introduced a number of standards and built some packages to try and enforce those standards. So there's the standard for how we, on the server side, get configuration from the environment."

Another example of a custom tool is a script that checks whether an agreed development standard to achieve security has been complied with, as a manager at Alpha described:

> "so, there's a lambda [function] that runs when you make a bucket, it triggers and goes 'you didn't encrypt,' it turns on encryption, tells you, 'you are an idiot.' "

The decision on whether to write a custom tool or customize an existing tool comes with the question as to whether the effort involved can be sufficiently carried by the organization. This is particularly true if off-the-shelf or open-source solutions exist.

Capturing NFRs in version control The fourth and final management practice that we identified is codifying and capturing an NFR in version control. Codification refers to capturing knowledge of NFRs in source code files and other artifacts, such as IaC configuration files, which can then be managed in version control systems. Configuration for tools (including static analysis tools, code linters, and formatters) can be stored in source control alongside the source code. The potential configuration of an NFR could include rules or triggers that represent thresholds for certain metrics that could trigger alarms or failures through a dashboard, or the CSE pipeline itself.

Codification helps set objectives for developers who might not have had enough time to gain the tacit knowledge about what constitutes an acceptable definition of an NFR. Knowledge of NFRs may not be easy to communicate via natural language, as one developer at Gamma described:

> "with respect to codifying something vs documenting it: it's not precise enough [if] it's written in English. It's open to interpretation."

Similarly, as for any requirement, NFRs must be specific, reducing any room for interpretation. For example, stating that "the response time must be fast" is not specific; instead, "the response time must sub-100ms" provides clarity that can be captured in source control.

We found that the management practices we identified through this study were primarily used in response to some triggering event. That is, triggering events would reveal a lack of shared understanding, and if sufficiently severe, companies would take action to address it to prevent it from happening again. Why are these practices primarily used reactively, rather than proactively? One reason is that management practices, in general, tend to emerge due to an organization's realization and increased awareness. For example, Gawande (2010, p. 33). described how the pre-flight checklist emerged in response to an airplane crash in 1935: the pilot had forgotten to conduct certain procedures in a relatively new airplane. The complexity of software development in a fast-paced CSE setting represents a challenging context for developers and other stakeholders to maintain a shared understanding. The management practices can be seen as 'checklists,' activities and practices that address specific issues that had emerged at an earlier point in the project. It is not until one has awareness of a problem, that it can be addressed. The practices described above therefore reflect a *response* to an increased awareness. A proactive use of these practices would imply prior knowledge of problems before they would emerge. While some problems could be anticipated, it is more likely that issues with NFRs emerge only after initial implementation.

4.2.2 Communication Practices

A second set of practices to build a shared understanding of NFRs are communication practices. Proposition 3 states that these practices are primarily used proactively, without a triggering event. We discuss possible reasons why this is the case after we lay out the practices we identified.

Soliciting Feedback The first communication practice we observed was soliciting feedback. Feedback activities such as code reviews and deep-dives as part of the CSE pipeline process, developers and other stakeholders can explain and clarify NFRs, and any issues or misunderstandings can be identified and addressed. A developer at Delta explained: "there are things like code review, review and feedback from the managers that sort of catches the NFRs." A code review is a common and proactive (i.e., not triggered by events such as those listed in Table 5) way to build a shared understanding of an NFR and also to solicit feedback that a given code contribution satisfies the expectations for some NFRs. A thorough code review may reveal when expectations for an NFR have not been met. For example, code reviews can reveal the testability of a code unit, or when unit tests do not provide complete code coverage.

Other types of feedback meetings such as deep-dives, also create opportunities for software teams to proactively build a shared understanding of NFRs. Software developers can discover and initiate the shared understanding of NFRs as one developer at Delta explained:

> "our [deep-dive] during the development process and the project engineering role really helped with that. Because when the shaping is happening, we're starting to think about those non-functional requirements: how it might work for other use cases, how something should be developed from a quality perspective and what tools we are using."

Software teams can use deep-dive meetings to demonstrate a product or a design, and subsequently discuss and verify that the product is aligned with both low-level requirements and high-level requirements.

A shaping meeting occurs at the start of a project where project stakeholders decide on the initial high-level requirements of a planned product. A deep-dive meeting, in contrast, occurs after project implementation begins, and usually includes only software developers or managers who are directly involved in the implementation. Continuously soliciting feedback on NFRs allows developers and other stakeholders to share and build knowledge regarding NFRs, contributing to a shared understanding.

Leveraging Past Experience The second communication practice to build a shared understanding of NFRs is to leverage past experience. Previous projects may be reviewed, allowing developers to reflect and deepen their understanding of how NFRs were handled. Reflecting on past experience and discussing this is thus a communication practice. A developer at Delta indicated this to be an important practice:

> "I think the biggest tool that has helped me learn NFRs has really just been reviewing previous work, specs, and integrations."

Unlike other communication practices, leveraging past experience was primarily a reactive practice; triggers such as technical debt or new requirements would encourage developers to address their lack of knowledge of the NFRs they focused on. However, this practice could also be undertaken proactively; for example, a developer at Delta described:

> "there's a process [to learn about NFRs] that isn't formally presented, it's picked up over time through training. I think that the majority of NFRs have been picked up that way."

An organization may proactively incorporate this practice in other processes such as the onboarding process, to have dedicated training and ongoing support (Sharma and Stol, 2020).

Wireframing The third communication practice we identified is wireframing, which includes the use of visualization tools to create designs, mock-ups, demos, or sketches to describe NFRs and improve developers' understanding of NFRs. This is consistent with Wagner et al. (2019)'s theory of requirements engineering; wireframing is a form of prototyping, which helps to promote shared understanding of requirements among stakeholders. Teams may proactively visualize an NFR, such as usability, maintainability and interoperability. A developer at Delta described their use of tools such as Figma or Figjam "to quickly visualize what we're talking about." Sharing a wireframe or mock-up that describes an NFR can be a useful way to come to a shared understanding of an NFR and elicit feedback from stakeholders. A wireframe can serve as a common reference for developers on a project to communicate expectations of an NFR, such as usability. Due to the fast pace of change inherent in CSE, developing a wireframe and mock-up may help project stakeholders build and maintain a shared understanding of an evolving NFR. Teams can also share mock-ups and minimal software designs with stakeholders during meetings to get feedback to determine expectations for usability, as a manager at Delta explained:

> "that's where things like 'shaping', wire-frames, starting to try to think through that user experience really helps. Design drawings, I find are really helpful. Early mock-ups help people because I find that a lot of the team does well with actually experiencing something."

Knowledge-sharing activities The fourth communication practice to support the building of a shared understanding of NFRs is the use of knowledge-sharing activities. These include 'guild' meetings or other regularly scheduled meetings. Organizations can create a skill-focused group, such as a guild, to centralize individuals working on specific aspects of projects and encourage knowledge sharing amongst and within these groups (cf. Stol et al., 2022). A guild may include individuals from diverse technical and non-technical teams within an organization, enabling cross-functional communication and collaboration between project stakeholders (Cooper and Stol, 2018). Discussions through a guild allow an organization to proactively focus on a specific important NFR; for example, a release management guild may organize exercises or training for communicating configuration and software deployment expectations that may affect the availability or portability of the software. A developer at Delta described their experiences: "it's been helpful for talking about how we build things and non-functional requirements in specific areas of our jobs." Discussing problems and solutions within a guild can also lead to the creation of development standards that focus on specific NFRs (e.g. security), as discussed earlier. A manager at Delta described this experience:

> "Great user experience is one of those things that's difficult. When it comes to user experience, we've centralized the people who make user experience decisions into a group and then we have standards in that group."

Such development standards for NFRs remove ambiguity and ensure that each software team member understands what constitutes good test coverage and ease of unit or regression testing.

Asking questions The last communication practice we identified was asking questions. Asking questions frequently during a project enables building a shared understanding of NFRs among project stakeholders. We found this to be a popular approach for building a shared understanding of NFRs, in particular during the early stages of the software development process. Developers may use question-asking to clarify expectations for an NFR and ensure that each stakeholder has a good understanding of a particular NFR. Software developers and managers may ask a client questions to elicit the NFRs that are important during the early stages of a project. A manager at Delta described:

> "you want to ask a lot of questions, you want to poke a lot of holes and things and, really go to the very edge of, you know, when you're sketching it out."

During a meeting or conversation, software developers and other stakeholders may develop a common understanding of an NFR even with a minimal set of questions being asked. A developer at Delta described:

> "when we go into the deep-dive, we know when the non-functional requirements have been met when we as a group meet, and there aren't gaps in our shared understanding. We come out of a review meeting, and there's not a lot of feedback."

The effectiveness of this practice depends on the richness of the communication medium. Asynchronous channels, including dedicated message relay systems such as Slack, may lack the bandwidth necessary for a detailed explanation of the expectations around an NFR. In contrast, synchronous channels, such as video calls or in-person conversations, offer considerably more communication bandwidth.

The communication practices that we identified were primarily used proactively, though the leveraging of past experiences was also used reactively, in response to issues with a specific NFR. Why are communication practices primarily proactive, as opposed to the primarily reactive management practices? One reason might be that all communication practices fit more naturally with the daily activities of software developers, from onboarding and training of software developers to design and implementation. Leveraging past experience and wireframing, for example, aligns naturally with software design activities. Knowledge-sharing activities and asking questions fit well within the daily professional activities of software developers (Kautz and Kjærgaard, 2008; Meyer et al., 2021). Management practices, as discussed above, on the other hand, are primarily reactive because they address specific issues that arise; the solution to those issues is not so much communicating them to others, but rather ensuring that these issues do not resurface once solved.

4.2.3 Different Non-Functional Requirements Require Different Practices

Proposition 4 states that different non-functional requirements require different practices for building shared understanding. As discussed in Sec. 2.2, NFRs are frequently seen as a category of requirements, contrasting them with functional requirements. Embedded in this categorization appears to be an assumption that these NFRs share certain characteristics and that they can be addressed and dealt with in similar ways, for example, how architects "consider" NFRs (Ameller et al., 2012a), or that agile methods can be tailored to "handle" NFRs (Rahy and Bass, 2022).

If we link the practices in Table 6 to specific NFRs that were discussed in the context of the various exemplar quotes, a different picture emerges that reflects the variety in

nature of NFRs. It is intuitively clear that an NFR such as security is quite different from usability or maintainability.[7] For example, to build a shared understanding of scalability and performance, two NFRs that are arguably very closely related, companies used the practices of quantifying and offloading the NFR. To develop a shared understanding of these two NFRs, their ability to be quantified helps because writing scripts and code to monitor, set thresholds and cut-off values to trigger alarms, etc. is a way to capture an understanding of these NFRs, which subsequently can be shared among different team members. This practice, however, would be far less useful for building a shared understanding of usability. A different practice, such as wireframing, is likely more useful for elements of a system that need to be visualized.

We note that all management practices we identified (see Table 6) through this study were in relation to one or more specific NFRs. For example, quantifying an NFR was used for scalability (at Alpha) and performance (at Beta and Gamma). Other practices were also observed in relation to specific NFRs. Several of the communication practices, on the other hand, appear to be sufficiently generic that we deemed them suitable for 'all' NFRs. For example, leveraging past experience and asking questions are two practices to build a shared understanding that could be applied to any NFR.

It is worth noting that companies may use other practices than those listed in Table 6, or that these practices may be used for building a shared understanding of other NFRs not in the table. Therefore, Table 6 should not be read as an exhaustive catalogue of practices for building shared understanding of NFRs.

Why do different NFRs require different practices for building shared understanding? While the software engineering literature frequently considers NFRs as a class of requirements, contrasting them with functional requirements, this classification is limited, as discussed in Sec. 2.2. The SE literature commonly considers the "ilities" of a software system, emphasizing the fact that they are typically cross-cutting across the whole system, more so than functional requirements. While this is true, there has been little attention within the SE literature for distinguishing various NFRs, with a resulting tendency to treat all NFRs alike despite the fact that they differ quite significantly. Usability, for example, is quite different from security, even though these NFRs can be closely intertwined in practice. For example, frequent two-factor authentication requests degrade users' perceived usability but can increase overall security. As noted in Sec. 2.2, performance is also an NFR that borders on becoming a functional requirement when it is lacking. A developer at Beta characterized this as follows:

> "I would say performance is never a high priority until somebody says 'it doesn't work for me,' in which case you've graduated from an NFR to, 'it is not functioning for me.' "

Thus, while NFRs as a class of requirements are frequently considered in discussing software architecture and architectural styles (Harrison and Avgeriou, 2007; Lundberg et al., 1999), the fact that they are so different means that it is likely that specific practices are necessary for each, in support of Proposition 4. This observation that not all NFRs are equal is also consistent with several findings in recent work on requirements engineering; a recent exploratory study by Nasir et al. (2023), for example, found that practitioners do not have well-defined procedures to specify different types of NFRs, and more generally,

[7] Maintainability, for example, is an attribute relevant at development-time, but does not imply a run-time attribute.

findings seemed to suggest that practices varied across different NFRs. Wagner et al. (2019) found that NFRs may be quantified or not, depending on the NFR.

4.3 Continuous Software Engineering as a Source of Challenges

The literature on CSE suggests that ongoing customer feedback and rapid iteration cycles should increase the level of understanding of customer requirements (Bosch, 2014; Fitzgerald and Stol, 2017; Klotins et al., 2023). Similar to agile methods, which were developed based on the realization that getting feedback from customers early on in the process is beneficial (Williams and Cockburn, 2003), CSE practices also suggest that learning through fast iterations can help to understand what users and customers need and use. For example, using the practice of continuous experimentation (including A/B tests) allows development teams to collect evidence on how different variants perform on certain criteria that the team are interested in (Quin et al., 2024). Through feedback, learning, and empirical evidence rather than guesses, team members could develop a better shared understanding, including NFRs.

The evidence from our study suggests a different picture. We found that the nature of CSE in small organizations is a source of challenges. For example, we found that different organizational departments and roles did, in fact, exhibit a lack of shared understanding of NFRs. Our fieldwork identified a number of challenges. For example, the pace implied by CSE practices actually inhibits building a shared understanding of NFRs. Hence, we posit that CSE, while emerging as a mainstream software development approach (Bosch, 2014; Klotins et al., 2023), also comes with considerable challenges in the context of maintaining a shared understanding of NFRs. Or, more formally stated:

Proposition 5: *The characteristics and context of continuous software engineering in small organizations make building a shared understanding of non-functional requirements challenging.*

Table 7 presents selected evidence in relation to a number of challenges that we linked to the nature of CSE environments in small organizations.[8] These challenges were identified through our fieldwork at the four small case organizations, but it is likely that this list is not exhaustive. We discuss each challenge in detail.

Table 7 Challenges due to continuous software engineering in small organizations

Challenge	Example	Description	Company
Fast pace leads to deprioritization of NFRs	"Some of the core pieces of the system again get more love or more time to knock that [technical debt] number down or we just pay closer attention"	Fast pace with limited resources in small organizations means that not all parts of the system receive the same level of attention and care.	Alpha
	"we tend to deprioritize ones that aren't like hard functional problems"	The complexity of NFRs tends to be higher than FRs, as the latter translate more readily to specific functions or features in a system.	Beta

[8] The set of challenges in Table 7 is a revised set of challenges that were previously reported in Werner et al. (2020, 2022).

Table 7 Challenges due to continuous software engineering in small organizations (continued)

Challenge	Example	Description	Company
	"You build an MVP and there's always going to be room for a reorganization later on as you better understand your problem space."	An initial implementation (MVP) is focused on delivering functionality quickly rather than ensuring that NFRs are achieved.	Gamma
NFRs are hard to automate	"[tests] always [had] a bias toward the happy cases [...] When something does go wrong, it's something horribly out of left field [*the untypical, unusual (ed.)*] How do you prepare for those? How do you think about what left field is?"	CSE emphasizes automation, but not all NFRs can be automatically tested.	Gamma
Lack of domain knowledge	"the scoping and the understanding of how broken down you need to have a ticket with descriptions, to understand the non-functional requirements that need to happen"	Small organizations by definition have fewer staff and thus more limited in the collective memory or knowledge base. This is particularly challenging if small companies need to pivot to new domains.	Beta
Inadequate communication	"The thing is, non-functional requirements are not always communicated as much, trying to figure out what or not, understanding or communicating expectations around some of those NFRs from stakeholders."	Time constraints in small organizations can preclude sufficient communication, leading to a divergence and decay of shared understanding.	Delta
	"the root of it is we built one half (front-end) without building the other (back-end)"	Timeliness of appropriate communication is necessary but may not happen due to time constraints or mismatched priorities in small organizations.	Beta
Knowledge of NFRs is not documented	"I try not to get hit by a bus. So does [colleague's name]. Certain parts of the system are maintainable because certain people know how they work versus it being [explicitly] documented."	A tendency to not document everything, partly due to a lack of time and resources, means that key information is held by a small number of individuals, which means that such knowledge cannot contribute to a shared understanding.	Alpha
	"at the moment it is tacit knowledge and unfortunately when a new developer comes in and starts working on stuff [they struggle]"	Building shared understanding among new developers is challenging due to a lack of documented knowledge.	Beta
Loss of control	"We do the easiest thing to get ourselves up and running, but in the long term, we have additional overhead, which isn't critical to a startup, as we didn't have the knowledge nor the resources to hire the people with the knowledge"	While initial offloading of NFRs requires an understanding of NFRs, such shared understanding may decay over time if not maintained among the team.	Gamma

Table 7 Challenges due to continuous software engineering in small organizations (continued)

Challenge	Example	Description	Company
	"Availability is actually an interesting one, do you spin up Amazon services in Virginia, or do you avoid that because that's their biggest region and so things sometimes go wrong there."	Offloading an NFR, such as availability, must be consciously done with some understanding of the provider's infrastructure. As this example illustrates you may not want to have all of your services in the same region within Amazon, especially if it is the largest region within Amazon's environment. On the flip side, however, this might also mean that if there is an issue Amazon may quickly react.	Alpha

Fast pace leads to deprioritization of NFRs. The first challenge is that NFRs tend to get deprioritized due to an emphasis on fast iterations. As CSE seeks to deliver new functionality quickly, minimizing the time between request and delivery (Fitzgerald and Stol, 2017), there is a tendency to prioritize functional requirements, as succinctly shared by one developer at Beta: "we tend to deprioritize ones that aren't like hard functional problems." This emphasis on functional requirements at the expense of NFRs has previously been observed in studies of agile methods as well (Ramesh et al., 2010). The increased emphasis on 'speed' and 'faster iterations' of CSE increases the risk of deprioritizing NFRs (Gralha et al., 2018; Savor et al., 2016). The lack of resources that is prevalent in small organizations appears to exacerbate this; for such organizations that seek to deliver fast, removing an unclear, ill-defined NFR from a sprint in order to maintain the pace seems to be an easy choice. Consequently, a shared understanding of NFRs is subject to becoming quickly outdated. A manager at Alpha described this focus on a fast release of new features at the cost of building up technical debt:

> "When you're first starting out it's move fast and break things; get things out the door. There's fallout, could be minimal fallout, but something you take a chance on."

Several organizations admitted to not performing adequate testing of either functional or non-functional requirements, citing a lack of resources that was inherent to the small scale of these organizations. A developer at Gamma recalled:

> "we weren't doing any kind of testing, A/B testing, or anything like that to assess performance or whether these changes were having a positive or negative impact."

NFRs are hard to automate Related to this is the challenge that some NFRs are very hard to automate. CSE emphasizes a fully automated build, test, and delivery pipeline (Fitzgerald and Stol, 2017; Fowler, 2024; Klotins et al., 2023). Without appropriate attention to automating validation and verification of NFRs, these may become a bottleneck to the software delivery process. One factor here is the writing of suitable tests; it is often easier to write test cases for the normal or "happy path," i.e., test cases that test the normal operation of the software. Anticipating what might go wrong with different NFRs is more challenging, which means writing tests that anticipate abnormal behaviour is harder, and therefore less likely to be done. A developer at Gamma summarized this lack of focus on the unusual path (referred to as 'left field') as follows:

> "[tests] always [had] a bias toward the happy cases [...] When something does go wrong, it's something horribly out of left field [...] How do you prepare for those? How do you think about what left field is?"

Lack of domain knowledge Another challenge that is more likely to occur in small organizations is a lack of domain knowledge. Large organizations tend to have more organizational memory, allowing developers to draw on the domain knowledge of colleagues; this may not be the case in small organizations (Richardson and Von Wangenheim, 2007; Sánchez-Gordón and O'Connor, 2016), in particular start-ups that are growing and tend to focus on one specific domain of interest. If several new staff members are new to a given business domain, they will simply not have a shared understanding of the issues at hand. As small organizations grow, they may seek to apply their technologies to other domains, implying that they, once again, struggle with a lack of domain knowledge.

Some NFRs, such as security and privacy, may be related to laws and regulations (e.g., GDPR mentioned earlier), which may be difficult to understand and interpret by software professionals. Specific standards and guidelines for certain regulatory domains, such as DO-178C for aviation and IEC-26262 for the automotive domain, may also be a source of certain NFRs. Dissecting and disseminating regulatory information requires careful coordination between staff who can understand the jargon and staff who need to implement related NFRs.

A lack of domain knowledge can also negatively affect the priority that should be placed on developing a shared understanding of an NFR. An organization may exhibit some disparity in the priority placed on NFRs, or even a single NFR, especially between two units within an organization. While an experienced developer may place heavy emphasis on certain NFRs, these may not be fully appreciated by non-technical teams, such as marketing or sales, who subsequently perceive such NFRs as having low priority.

An NFR may have such a low priority that it receives no attention at all, until it causes a partial or total loss of functionality, at which point an organization has no choice but to elevate the priority of the NFR and assign dedicated resources. In such a case, an organization is likely to rely on experienced developers, who in effect become a bottleneck for the organization in maintaining the pace of software deliveries in continuous software engineering. The risk of experienced developers departing from the organization then becomes a major risk.

Inadequate communication A fourth challenge we identified in the context of CSE in small organizations is insufficient communication among developers and stakeholders. The cross-cutting nature of NFRs means that developers working on different parts of a system need to communicate regularly. For example, at several of the organizations, developers were found to be simultaneously working on the same problem, independently and in isolation, but not communicating the right information. This, in turn, leads to a lack of shared understanding. A manager at Gamma recalled this as follows:

> "two different developers building the front-end and the back-end at different times. Only like a week or two apart, but they weren't communicating well enough to each other and it wasn't enough like top-down planning of that."

We found a similar instance of this practice at Beta, as one developer recalled: "the root of it is we built one half [front-end] without building the other [back-end]." A lack of communication could be detected at a later stage, for example during code review as one

developer at Alpha mentioned: "... then somebody gave them a code review or whatever and said, oh you should have done it this way."

A lack of communication is a known challenge in software development in organizations of any size. In large organizations, lack of communication can be due to a lack of visibility and isolation due to imposed organizational structure: developers may simply not be aware of one another (Stol et al., 2014). In our study of small organizations, we found that a lack of communication is due to time constraints, but also time-sensitive. For information to propagate through an organization, even small ones, and develop into shared understanding takes time, in particular, if this information is not documented. As a developer at Gamma lamented: "there was no documentation about it." Developers who simply press on might make the false assumption that everyone else has the *same* understanding of a specific NFR. A developer at Beta recalled this case of false shared understanding:

> "there have been times where I've had a conversation with [person] about what a product needs to do and how he has envisioned it. And I guess some things just maybe got lost in translation. And I end up working on something that is different or works differently than what he had imagined. And vice versa."

Knowledge of NFRs is not documented A fifth challenge is that knowledge of NFRs remains undocumented. Tacit knowledge of NFRs means that it is difficult for newly joined staff to develop a shared understanding with their colleagues, as one developer at Beta described: "at the moment it is tacit knowledge and unfortunately when a new developer comes in and starts working on stuff [they struggle]" The continuous, iterative nature of CSE that comes with a pressure to deliver new features exacerbates this; in small organizations in particular, products may be new and initially delivered as a minimum viable product (MVP). One manager at Gamma described: "you build an MVP and there's always going to be room for a reorganization later on as you better understand your problem space." At small organizations, in particular, key knowledge of NFRs is held by only a small number of people at an organization, such as the founder and first employees who were involved in the initial architecture design. Consequently, the 'bus factor' poses a bigger risk in such settings, as the departure of key staff may immediately lead to a gap in shared understanding of NFRs. A developer at Beta described:

> "I know a lot of the team leads have a ton of non-functional requirements in their head and how things should work and they're kind of the ones gating[9] what goes out based on those undocumented non-functional requirements. If we were to document those we could get those ideas into the heads of the people actually writing the code and better the development experience."

Loss of control The last challenge we identified is the loss of control over an NFR as a result of offloading it to a third-party cloud provider. While offloading can help in satisfying certain NFRs, such as reliability and performance, it does come with a loss of control over an NFR, which subsequently negatively affects the level of shared understanding of that NFR within the organization. Without maintaining a certain minimum level of shared understanding of such offloaded NFRs, organizations may become too dependent or lose all shared understanding of these NFRs, jeopardizing the

[9] A manual process whereby those with the NFR knowledge are required to approve a particular change prior to deployment.

maintainability and viability of their product in the future. If something were to happen to an offloaded NFR, then the knowledge of how to rectify these issues should remain with the organization.

Why does the context of CSE in small organizations make building a shared understanding of non-functional requirements problematic? Even though CSE emphasizes rapid delivery, automation of processes, and increased collaboration among different organizational units (cf. DevOps (Fitzgerald and Stol, 2017)), there is an inherent chasm between business managers and development units that remains difficult to bridge—Fitzgerald and Stol (2017) proposed the concept of "BizDev" to address this, mirroring the DevOps concept. Product managers, in particular, "ceded control" of NFRs to developers, and relied on intuition to verify whether an NFR was achieved, as one manager at Beta suggested: "we're going to see this thing before it goes out the door, and we'll know something feels wrong in terms of a NFR." The different types of expertise needed by business managers and developers may also be a reason that a true shared understanding among different stakeholders is difficult to achieve. Further, small organizations tend to be low on resources and organizational memory; there is less staff than in large organizations, who will have less time to actively build shared understanding, either through communication or documentation.

4.4 The Ongoing Pursuit of Shared Understanding of Non-Functional Requirements

Creating a shared understanding requires a considerable investment, which may not be within reach of all organizations, nor may it be appreciated in the first instance by organizations. Organizations may not value shared understanding of NFRs, which implies a role for education and increasing awareness. In fact, we found that there was value for the four case organizations in conducting this study, as they increased their awareness of the importance of shared understanding. For example, a developer at Alpha, making reference to the focus group study we conducted, described:

> "the discussion revolving around the task actually created a shared understanding and let the developer to actually add a little description."

In follow-up conversations with informants, they noted that they experienced their level of shared understanding of NFRs as being "fluid"; this would manifest as a belief that they did in fact have a shared understanding of NFRs, at which point a problem might arise, such as a drop in performance, which required further investigation to rebuild shared understanding. Thus, at times, the line between shared understanding and lack of shared understanding may be fluid (an 'ebb and flow'), and ambiguous. Maintaining a shared understanding of NFRs is as arduous, perhaps even more than creating the shared understanding in the first place, especially as an organization grows and evolves. In that process, new staff get hired, others retire or leave the organization, and products and markets evolve. These factors tend to lead to a 'decay' of shared understanding, and so we posit that organizations must continuously actively pursue shared understanding of NFRs. Thus, we suggest that in tandem with the process described by Proposition 1, which suggests that a lack of shared understanding becomes apparent through triggering events (for example, a drop in performance as mentioned above), we argue that the 'level' of shared understanding of NFRs is not static, but rather dynamic and constantly fluctuating. Or, as a corollary to Proposition 1, more formally:

Proposition 6: *There is a continuously fluid boundary around the state of shared understanding of non-functional requirements.*

5 Toward a Theory of Maintaining Shared Understanding of Non-Functional Requirements

The interrelated set of six propositions presented in Section 4 together represent what we label the Iceberg theory (see Figure 2). The Iceberg is a metaphor (Stol, 2024) for a shared understanding of an NFR, with a portion of it 'above the water line' and the remainder invisible, below the surface. The shared understanding that is visible is constantly at risk of 'sinking' (decaying) if members of the organization are not actively maintaining it. Once shared understanding decays into a lack of shared understanding of NFRs, it sinks below the water line. At that point, it may or may not be detected; without proactive assessment, it takes triggering events to detect that there is such a lack of shared understanding.

The Iceberg theory integrates three main processes that describe a cycle, although there is no specific starting point. The first process refers to the realization that there is a lack of shared understanding through one or more critical triggering events (Proposition 1). The second process refers to action undertaken by an organization to address this lack of shared understanding (Propositions 2, 3 and 4). The third process that takes place is the emergence of challenges that lead to a decay of shared understanding due to the context that small organizations using continuous software engineering approaches face (Proposition 5), as well as the fluid boundary around the state of shared understanding of NFRs, represented by the wavy line between the lack of shared understanding and a true shared understanding (Proposition 6). The Iceberg theory describes how small organizations that use continuous software engineering approaches maintain (or struggle to do so) a shared understanding of NFRs.

In the remainder of this section, we ask: is the Iceberg theory a theory? (Section 5.1). What confidence can we have in the Iceberg theory? (Section 5.2). We then lay out implications for research (Section 5.3) and practice (Section 5.4).

5.1 Is the Iceberg theory a theory?

There is no lack of philosophical treatises on the topic of what constitutes 'theory' and one might question whether the Iceberg theory is, in fact, a theory. Given the lack of agreement across scientific disciplines in general (Mintzberg, 2017; Sandberg and Alvesson, 2021; Stol, 2024), let alone within the software engineering field, we briefly declare our position. Specifically, we adopt Sandberg and Alvesson (2021)'s argument that knowledge can be called 'theory' if seven structural elements are present, which are discussed below in more detail: purpose, phenomenon, conceptual ordering mechanism, relevance criteria, empirics (data), intellectual insights, and boundary conditions. In what follows, we describe these seven elements and how the Iceberg theory embodies these.

Sandberg and Alvesson (2021) present a typology of theory types (which goes beyond the more fine-grained distinction between process and variance theories), distinguished by the seven structural elements mentioned above, acknowledging that the theory types are not mutually exclusive and that some overlap may exist across these

theory types. The knowledge presented in this article is what Sandberg and Alvesson (2021) have labelled an 'enacting' theory. The purpose (element 1) of enacting theories is "to articulate how phenomena are continuously produced and reproduced: that is, the processes through which they emerge, evolve, reoccur, change and decline over time" (Sandberg and Alvesson, 2021, p. 502). The phenomenon (element 2) in an enacting theory is not something "as mostly given" (Sandberg and Alvesson, 2021), but instead something that is "constantly in the making and in progress" (Sandberg and Alvesson, 2021); Sandberg and Alvesson cite Latour (1986)'s example of 'society,' which is not something discovered by scientists. The building of a shared understanding of NFRs and its decay over time constitute such a phenomenon. An enacting theory provides conceptual ordering (element 3), not by relating a set of variables to one another such as in variance theories (a type of explaining theory (cf. Stol et al., 2022)), but rather by describing the processes that give rise to a phenomenon. The relevance criterion (element 4) of an enacting theory is whether or not the theory is able to shed light on the key processes of the emergent phenomenon. Thus, in evaluating the validity and utility of the Iceberg theory, we might ask: does the Iceberg theory illuminate the processes through which a shared understanding is built, maintained, and decays?

The empirical data (element 5) presented for an enacting theory should be in support of the articulation of the processes describing the phenomenon. In this article, we laid these out in Section 4 (see Tables 5, 6, and 7), all of which we argue support our articulation of the theory. One might ask: is the current empirical foundation of the Iceberg theory sufficiently rich? In answering this question, we echo Miner (1980, p. 7): "any theory, regardless of the method of its construction and the extent of current confirmation, is provisional in nature. Theories are constructed to be modified or replaced as new knowledge appears." In other words, while the Iceberg theory represents our current understanding of maintaining a shared understanding of NFRs, it is well possible that further research modifies these insights.

The intellectual insight (element 6) of the Iceberg theory lies not in a series of causally related variables, as would be the case for a typical variance theory, but rather in a demonstration that the (re)productive processes "are configured and interacting differently than previously thought" (Sandberg and Alvesson, 2021, p. 503). Indeed, one of the intellectual contributions of the Iceberg theory is that it challenges the assumption that merely offering a set of practices to proactively detect and build shared understanding as a matter of routine will lead software companies to adopt those practices. At least in the case of small organizations, we found a different mechanism that leads to such detection.

Finally, we discuss a number of boundary conditions (element 7) of the Iceberg theory. First, given our research focus on small companies, it remains unclear whether the Iceberg theory, as propositioned in this article, holds true for larger organizations. Given the paucity of research in this area, evaluating the Iceberg theory in larger development contexts is one avenue for further research. Second, the set of cases all delivered web-based systems; this is an artifact from one of the selection criteria, namely, that the case companies used continuous software engineering practices. CSE practices are more easily adopted for the development of web-based systems, as opposed to, say, embedded software systems, which remains to be a challenge (Antonino et al., 2018; Dakkak et al., 2022; Mattos et al., 2018). Third, the Iceberg theory integrates several key concepts that we previously reported on, including triggers, practices for managing NFRs, and barriers that are induced by the CSE context. While these concepts are grounded in

empirical observations, the specific sets of triggers, practices, and barriers may vary across different settings. That is, studies of other small companies using CSE practices may yield other such triggers, practices, and barriers. Other instances of these categories may exist. As such, the contribution of this article is not so much the reporting of these triggers, practices, and challenges on which we have reported earlier (Okpara et al., 2022, 2023; Werner et al., 2020, 2022), but rather the theoretical integration of these concepts and the articulation of a series of propositions that link them together to describe a number of processes.

5.2 Evaluation

Having argued that the Iceberg theory is, indeed, a scientific theory, we now ask: how much confidence can we have in the Iceberg theory? And, how could we evaluate or test the Iceberg theory? In this section we address these two questions.

The theory is developed based on analysis of qualitative research; one advantage of using qualitative research is that it offers a more holistic view to obtain much richer and deeper data than what could have been achieved using, for example, survey research (Marshall and Rossman, 2014). However, findings from field studies are inherently limited in their generalizability (Stol and Fitzgerald, 2018). Through substantive and inductive theory development, we sought to mitigate this limitation; we generated propositions that are grounded in empirical data, leveraging empirical observations to theoretical generalizability as opposed to statistical generalizability.

To determine how much confidence we can have in the Iceberg theory, we adopt Lincoln and Guba's quality evaluation criteria to evaluate the proposed theory, including their recommended techniques to overcome the specified limitations. In particular, credibility, dependability, confirmability, and transferability are discussed in sequence (Guba and Lincoln, 1994; Lincoln and Guba, 1985), followed by a brief discussion of how one might test the Iceberg theory.

5.2.1 Credibility

Credibility evaluates how trustworthy and believable the results are given that a qualitative investigator is subjectively interpreting the perspectives of informants, as opposed to objectively documenting actual reality. A potential threat to validity here could be that our interpretation is wrong, based on a misunderstanding of the data. Given the inductive nature of the study, which does not seek to establish an absolute 'truth' but rather a viable interpretation that 'makes sense,' the development of the propositions could be misinformed, and not reflect the reality of the participants. To establish the credibility of this study, we adopted a number of practices to minimize this threat to validity, including prolonged engagement, persistent observation, peer debriefing, triangulation, and member checking.

Prolonged engagement ensures that investigators are able to spend sufficient time with informants, at an organization to carefully capture important contextual information, avoid misinterpretations, and cross-check intermediate findings, for example through debriefing, member checking, and persistent observation (described below). Furthermore, a prolonged engagement builds trust between members of the organization and investigators, adding to the credibility of the investigators' interpretations of the data. Through

prolonged and immersive visits a substantial knowledge base of context, culture, and a repertoire enables well-informed and accurate interpretations of case study data.

Persistent observation ensures sufficient in-depth studies are performed to uncover sufficient details. A significant amount of time was spent at each organization, amounting to roughly 4 months at each organization, allowing extensive observation. During this period, the investigators engaged with a diverse and numerous set of members of the organizations. Having more than one investigator present should help to mitigate research bias.

Peer debriefing seeks to ensure that a particular investigator's interpretations are not biased or misinformed. Peer debriefing was used throughout each interview, and each focus group was attended by at least two investigators. In addition, multiple investigators were involved in the open coding approach throughout the thematic analysis, to ensure the analysis was consistent.

Different forms of triangulation were used: across different investigators, across different sites (i.e. organizations), and different data sources. Triangulation helps to cross-check interpretations, initial and emergent findings, and discuss any divergent insights.

Member checking validates that the interpretation of the data is in fact a real representation of the participant's actual perspectives. Member checking was performed through feedback from participants and any finding that did not receive positive support was noted and subsequently dropped. While member checking was used for the intermediate results, the propositions and the resulting Iceberg theory were not the subject of member checking.

5.2.2 Dependability

The dependability of a study is concerned with the 'stability' of data, the question to which extent the findings are reliable, and the traceability of any variance in findings. While we recognize that qualitative analyses are unlikely to be exactly replicated by different investigators due to the fact that different researchers might have different training, theoretical sensitivity, and experience in both qualitative research and theory development, we adopted a number of best practices to achieve consistent understanding among the investigator team.

We used thick descriptions to describe the research method, circumstances, setting, context, and findings. In addition, we took detailed notes throughout the research process, including audio recordings (with the informants' permission) where possible through all in situ observations and interviews. The notes and memos, as well as the recordings, formed an audit trail for the investigator team to go back to the data, discuss intermediate findings, and discuss at length the interrelationships among the propositions.

Triangulation was used across investigators in the coding process, but also in the analysis process itself. That is, categories were identified as abstractions from multiple incidents (events, practices, challenges, etc.) that were mentioned by informants. Further triangulation across data sources (e.g. interviews, observation, development tasks) helped to ensure a consistent understanding of the data.

5.2.3 Confirmability

Confirmability evaluates the levels and variety of biases that prejudice research findings. Research is susceptible to different forms of bias (sampling, non-response bias, response, confirmation, and investigator bias); we discuss these briefly.

Sampling bias could represent a threat when selecting organizations to study. The companies we studied were contacted through our professional contacts, and they were local to the area where the lead investigator was based. While this allowed convenient access, the question is whether we would be able to observe similar findings in a set of organizations in a different area or country. However, we do note that the four organizations were not in touch with one another and that our findings across these four organizations were remarkably consistent.

Response bias was mitigated by anonymizing interviews and focus groups, developing rich relationships through in-depth knowledge and trust, and offering participants an opportunity to speak freely and in confidence. Multiple investigators were involved in all the data collection procedures, including the interviews and focus groups.

Confirmation bias was addressed by using open-ended questions encouraging respondents to provide their own points of view. Interviews were conducted in a semi-structured manner to allow extemporaneous questions, while making sure that our questions were not leading.

Investigator bias was mitigated by involving multiple investigators through each step of the research. In addition, open coding (Corbin and Strauss, 1990) was used to develop an inductive code book (Cruzes and Dyba, 2011), thus minimizing a coder's ability to enforce a bias towards any particular proposition. Finally, triangulation, peer debriefings, and member checking were all utilized to minimize any particular bias.

5.2.4 Transferability

A theory aims to be a generalization that provides an understanding of a given phenomena of interest. Note should be taken of the boundary conditions (see Section 5.1), lest the theory be "utilized fruitlessly in situations for which it was never intended" (Miner, 1980, p. 8). Transferability, then, is the assessment of the extent to which the Iceberg theory applies to other organizations. In this case, given our focus on small companies using CSE, we must first declare that the Iceberg theory only applies to those organizations using CSE practices. What remains is the question: does the Iceberg theory hold for medium and large organizations?

To allow assessing transferability we used two techniques: thick descriptions (Geertz, 2008) and purposive sampling. The purpose of thick descriptions is that "it creates verisimilitude, statements that produce for the readers the feeling that they have experienced, or could experience, the events being described in a study" (Creswell and Miller, 2000). Thick descriptions permit comparison of circumstances, settings, and findings used throughout this research, based on detailed notes and audio recordings. The second tactic, purposive sampling, refers to an investigator's judgment as a principle of selection of cases (Robson, 2002). We selected four small organizations that were using CSE practices, and which were also local and therefore easily accessible (Miles and Huberman, 1994). These criteria helped to collect highly relevant and meaningful data relevant to our research question.

Returning to the question: does the Iceberg theory hold for larger organizations? Most large organizations do not operate as a single entity, but rather as a federation of smaller entities such as teams, divisions, and business units, each typically with their own focus (Stol et al., 2011). For that reason, the Iceberg theory could also apply to contexts of these smaller organizational units. The extent to which this is the case, and how the larger organizational context in which these smaller organizational units are embedded would have to be assessed in more detail.

5.2.5 Evaluating the Robustness of the Iceberg Theory

We now address the question: how could we evaluate or test the Iceberg theory? When discussing theory testing, Popper (1980) is frequently cited for his idea of falsification; that is, Popper held the view that theorizing involves the development of statements in a form that allows the researcher to test them against observations in the real world (Gregor, 2006, p. 614). Arguably, the propositions of the Iceberg theory are falsifiable, but we argue that this is not an effective way forward for several reasons. First, Popper in effect sets constraints on what 'theory' is by demanding that all statements of a theory must be falsifiable. This view is far more narrow than Sandberg and Alvesson (2021)'s broader view that we adopt in this article, and also Gregor (2006)'s view, as she presented different types of theory. Falsification as a form of testing is primarily suitable for theories for explaining and predicting (Gregor (2006)'s Type 4 theory). Indeed, the Iceberg theory's purpose lies closer to that of Gregor (2006)'s Type 1, namely analysis and description. Second, the Iceberg theory is developed using an inductive research strategy, whereas Popper rejects induction as a valid method for generating theory. Thus, testing a theory using the method of falsification which itself rejects the method of developing the theory would be inappropriate. Third, the Iceberg theory is more closely aligned to Sandberg and Alvesson (2021)'s pluralistic and inclusive view of what constitutes theory, rejecting that there is a single universal standard such as Popper's view. Which criteria, then, should one use to judge a theory? This will depend on which philosopher of science we ask. Rather than asking how accurately the theory describes objective reality, pragmatists (such as Peirce, Dewey, and James) would subscribe to an instrumentalist view, namely that theories should be evaluated in terms of how useful they are. This means to focus on questions such as: Does the theory work in practice? Does it fit with experience? And, is the theory open to revision?

5.3 Implications for Research

A good theory is "useful" (Mintzberg, 2017), and helps direct future research efforts "to protect researchers from wasting time" (Miner, 1980, p. 8); the Iceberg theory suggests a number of open-ended questions and unexplored avenues that merit future attention (see Table 8). First, our fieldwork revealed a number of categories of triggers that manifest the lack of shared understanding of NFRs (Proposition 1). As mentioned briefly above, it is likely that these are not exhaustive, and that other types of triggering events exist through which a lack of shared understanding of NFRs can be detected. Future work can focus on other categories of triggers, or, identify other mechanisms through which a lack of shared understanding of NFRs is detected. A more important implication of Proposition 1 is that the theory challenges any notion that software organizations would

proactively use practices to detect a lack of shared understanding. Instead, our fieldwork suggests that a lack of shared understanding is revealed through emergent problems, which we labelled triggering events. One viable reason for this could be the general lack of resources (time, staff) in the small organizations that we studied. Whether or not Proposition 1 holds for large organizations remains an open question.

Second, we posited that what we labelled 'management' practices are primarily used to reactively build shared understanding (Proposition 2), and that a proactive building of shared understanding of NFRs is done through what we labelled 'communication' practices (Proposition 3). Future work can (a) identify other practices in either category, (b) identify other categories of practices if they are fundamentally different from these categories, and (c) evaluate these practices in terms of their effectiveness and efficacy. Other areas of future studies can study the differences between these different categories of practices. Is the proactive building of shared understanding as effective or primarily wasteful due to the lack of any critical event that triggered it? Is the resulting shared understanding as a result of reacting to a trigger more efficacious than proactively building? Further, we have argued that the differing nature of various NFRs requires that different practices are used to build a shared understanding of those (Proposition 4). Some NFRs are more readily quantifiable, whereas others are not. Management practices, in particular, which we argued are primarily adopted in response to an observed lack of shared understanding, appear to be tailored for each NFR (Proposition 3). Communication practices, on the other hand, appear to be more readily applicable to any NFR (Proposition 2). However, while we argue that both categories (management and communication) of practices are important, given that a lack of shared understanding is primarily revealed through triggering events, we posit that management practices that are used to address such lack of shared understanding in response to those critical events should take priority, also because addressing a lack of shared understanding of a specific NFR requires tailoring to that NFR (Proposition 4).

Third, we found that the specific context of small organizations adopting CSE approaches induces a number of barriers and challenges, which then contribute to the decay of shared understanding (Proposition 5). Future work can elaborate further on this process of decay in a number of ways, including an exploration of the boundaries of this theory. The Iceberg theory emerged through fieldwork at small organizations, which we argued are at particular risk of decaying shared understanding. Further, the organizations used CSE practices, and a further study of the interaction between the CSE paradigm (emphasizing continuous delivery, rapid cycles, and reducing intra-organizational barriers) and shared understanding over time is warranted. Given the increasing importance and adoption of CSE practices in the software industry today, replacing agile methods, we suggest this interplay between CSE and decay of shared understanding (and, arguably, 'knowledge' in a more general sense) appears to be an important topic. How fast does shared understanding decay? What are the main drivers of the decay of shared understanding of NFRs? Another direction for future work could investigate how to mitigate the various challenges that emerge in small organizations.

Fourth, the Iceberg theory argues that there is a continuously fluid boundary between shared understanding and the lack of it (Proposition 6). While the current study has identified a number of factors that lead to the decay of shared understanding, such as staff turnover, future work could focus specifically on this ambiguous and fluid boundary. Specifically, what factors contribute to this fluid boundary? Is this boundary more porous for some NFRs than for others? If so, for which ones, and why? The very classification

Table 8 Findings and implications

Proposition	Implications for research and practice
Proposition 1: A lack of shared understanding of non-functional requirements becomes apparent through triggering events.	The identified triggering events are unlikely to be exhaustive; future work can focus on other categories of triggers, or identify other mechanisms to detect a lack of shared understanding of NFRs.
	The Iceberg theory can raise the awareness of triggers as critical events that are manifestations of a lack of shared understanding. Without such awareness, these critical events may simply be ignored. Practitioners should therefore pay attention to these triggers, and investigate further when these events arise.
Proposition 2: Management practices are primarily used to reactively build a shared understanding of non-functional requirements, in response to triggers.	Future research could focus on identifying other management practices and determine whether these are indeed reactive (testing the Iceberg theory); and whether there are other types of practices beyond management practices that could be used reactively to build shared understanding of NFRs.
Proposition 3: Communication practices are primarily used to proactively build a shared understanding of non-functional requirements.	Future research could focus on identifying other communication practices, and determine whether these are indeed proactive (testing the Iceberg theory); and whether there are other types of practices that could be used to proactively build shared understanding of NFRs.
Proposition 4: Different non-functional requirements require different practices for building shared understanding.	Distinguishing management practices from communication practices helps in understanding the type of action to take. Awareness and familiarity with different practices, and their outcomes, helps in realizing that practices may not be suitable to address a lack of shared understanding of all NFRs: some practices may be specific to certain NFRs.
Proposition 5: The characteristics and context of continuous software engineering in small organizations make building a shared understanding of non-functional requirements challenging.	Further work could investigate the specific process by which the context of CSE in small companies leads to decay of shared understanding of NFRs, including causes, consequences, and potential mediating and moderating mechanisms. Further, future work can test whether or not the Iceberg theory would also apply in medium and large organizations. Cognizance of the challenges that small organizations face and the associated risks involved to decay of shared understanding of NFRs would suggest that training and increasing awareness may be good activities.
Proposition 6: There is a continuously fluid boundary around the state of shared understanding of non-functional requirements.	Future work could focus on the ambiguous and fluid boundary that the Iceberg theory proposes exists between shared understanding of NFRs and a lack of it. What factors contribute to the fluid boundary, what makes the boundary porous, and is this the same for all NFRs, or is shared understanding more sensitive to decay for some NFRs than for others?

of NFRs as a specific type of requirement, contrasted with a functional requirement, may be too simplistic as a dichotomy, as alluded to earlier. Several other scholars have made similar observations (Eckhardt et al., 2016; Glinz, 2007). NFRs affect every aspect of a software system, while some NFRs may be more superficial, others are pertinent to the success of the software. A large all-encompassing NFR, for example, performance and security, may be difficult to slice into pieces that can be implemented in a relatively short, rapid CSE cycle (Bellomo et al., 2014), thus further complicating actually building a shared understanding of this NFR.

Finally, as also mentioned as part of the boundary conditions of the Iceberg theory, whether or not the theory holds in larger than small organizations remains an open ques-

tion. Can similar processes be observed in large organizations? Do large organizations face similar problems in terms of the decay of shared understanding?

Further, going beyond shared understanding of NFRs, one might explore whether this cyclical process of creation and decay applies to other areas within software engineering practice. Closely related are, of course, knowledge management (Bjørnson and Dingsøyr, 2008) and architectural knowledge (Capilla et al., 2016). As Generative AI is quickly emerging as a pivotal technology that is changing the nature of software engineering (Stray et al., 2025), it will become important to understand the interaction between human developers and AI agents, and how this might change the nature of "knowledge" that we have of systems that are co-created with AI.

5.4 Implications for Practice

The Iceberg theory has a number of implications for practice. Organizations can leverage the theory in a number of ways. First, the theory draws attention to the topic of shared understanding of NFRs, a theme that has hitherto not received much attention. Familiarity with the theory may help in recognizing triggering events at organizations; in other words, the theory may help in determining that an organization has challenges that can be linked to a lack of shared understanding of NFRs (Proposition 1). The theory can help raise awareness of 'triggers,' and that the presence of such critical events (e.g. service disruptions) may be manifestations of a lack of shared understanding—without such awareness, managers and developers may simply ignore these manifestations and focus only on solving such critical events without knowing what might have caused them. This, however, would not prevent future potential problems.

Second, the theory offers practical guidance by describing a number of practices (Propositions 2, 3). While not exhaustive, they offer a starting point that organizations can use to build shared understanding. The theory distinguishes management and communication practices and also recognizes that not all practices may work for all NFRs (Proposition 4). Thus, the theory offers fundamental insights that can help organizations shape a strategy to address the lack of shared understanding of NFRs.

Third, the theory also helps organizations understand that their specific context will play a role in the decay of shared understanding, both through an understanding of the various challenges that can emerge (Proposition 5), and the realization that shared understanding is not a static asset in which to invest only once, but rather a dynamic pool of knowledge that requires constant maintenance (Proposition 6). While the impact of contextual factors is an area of potential future research (see Section 5.3), an awareness of their potential impact may prompt organizations to reflect on their specific context and investigate how their specific context might relate to critical events (triggers), whether they might trace these back to a lack of shared understanding, and if so, take appropriate action.

6 Conclusion

Despite the well-understood importance of non-functional requirements to a software project, exactly how small organizations build and maintain a shared understanding of NFRs in CSE is not yet well-understood. The current article concludes a research

program on extending our understanding of managing non-functional requirements in small software companies. An initial study in this program (Werner et al., 2020) identified a number of factors that contribute to a lack of shared understanding of NFRs in small-scale organizations using CSE. A subsequent article (Werner et al., 2022) explored how small companies using CSE manage NFRs, and what challenges the CSE context introduces. A further study (Okpara et al., 2022, 2023) explored how a shared understanding is reached in distributed development settings using CSE practices, as well as the challenges encountered in such a remote development setting. What has hitherto been missing is an integrated and holistic theoretical understanding of how these empirical observations tie together. Hence, we adopted an inductive theory development strategy (Stol and Fitzgerald, 2018), drawing on a rich set of empirical findings. The result of this effort is a middle-range process theory, labelled the Iceberg theory, comprising six interrelated propositions.

The resulting theory offers a novel perspective on how small organizations that have adopted CSE practices, maintain a shared understanding of the NFRs that are important to them. The theory captures three parallel processes. In the first process, triggering events raise developers' awareness that there is a lack of shared understanding of NFRs. In the second process, companies decide to invest in building a shared understanding of NFRs, either prompted (reactively) by a triggering event, or unprompted (proactively). In the third process, there is a continuous decay of shared understanding of NFRs, caused in part by the specific characteristics of continuous software engineering contexts in small organizations, as well as the continuous perturbations that software companies of any size may face, including staff turnover. In sum, the Iceberg theory constitutes a parsimonious explanation of a practical phenomenon in software engineering practice. We envisage the theory as "a node" as a kernel that can attract further knowledge (Miner, 2015, p. xi).

A Summary of Data Collection

Case study research is suitable for studying socio-technical phenomena within a real-world setting (Yin, 2002, p. 1). The case study approach afforded a prolonged engagement lasting several weeks or months onsite to conduct observations, interviews, and data analysis, as well as a level of protection from "missing instances, or over-simplifying and over-rationalizing process" (Ralph, 2019). The fieldwork allowed embedding in a realistic context as opposed to, for example, survey research which is far less amenable to capturing any realistic context (Stol and Fitzgerald, 2018).

A.1 Site Selection

A key criterion for selecting a case study organization was its relevance to the research (small companies using CSE), the ability to produce rich data and credible explanations, and be practically accessible (Miles and Huberman, 1994). We selected four local companies: Alpha, Beta, Gamma, and Delta (see Table 9). These companies were identified through personal contacts and selected based on accessibility and the companies' interest in the topic. Following the research protocol approved by the institutional ethics review

board at the University of Victoria, and non-disclosure agreements signed with the four organizations, the names of each organization and interviewees are anonymized. Each organization has grown from a small startup into a mature, established leader in its respective business domain, and adopted a continuous software engineering approach. The case companies were 6–10 years old at the time of the study, and each had a workforce of ca. 30 to 60 staff at the time of study.

All four companies delivered web-based business solutions but in different domains. Alpha delivers a system to support the collection and analysis of large volumes of data. Beta operates an e-commerce platform for its globally dispersed customer base. Gamma delivers an online content management system that includes online advertising. Delta delivers a system to manage loyalty, rewards, and referral programs.

We note that besides the domain of business applications studied, producers of other types of software systems were not considered in this study including computer game producers and embedded software organizations. Games (a type of 'casual' software (Meyer, 2013)) tend to have a single release date, not intermediate deliveries. Likewise, we did not study embedded software (a type of 'acute' software (Meyer, 2013)); implementing CSE practices for embedded software systems remains an unsolved goal. The focus on business applications, delivered through online platforms, therefore limits the scope to which the resulting emergent theory we develop in this article.

We now provide brief descriptions of the four case companies.

Alpha Company Alpha operates in the data crowdsourcing industry, collecting large amounts of data daily. The data is anonymously collected by geographically distributed data centres directly from devices spread out across the globe, including geographic information used to classify the data. Key NFRs for Alpha are configurability, security, and scalability.

Beta Company Beta offers an e-commerce platform for customers distributed worldwide. Their product is embedded into various platforms to seamlessly provide customizable booking and payment capabilities for non-traditional data types to various organizations. Key NFRs for Beta are usability, performance, and reliability.

Gamma Company Gamma is an online content provider, including advertisements. The platform receives millions of requests every second, each providing a customized responded based on various parameters as input. Gamma is able to track and tweak their platform by analyzing, in real-time, metrics and tweaking various parameters. Key NFRs for Gamma are performance, reproducibility, and configurability.

Table 9 Data set underpinning the development of the Iceberg theory. Cases Alpha, Beta and Gamma were previously reported in Werner et al. (2020) and Werner et al. (2022). Case Delta was previously reported in Okpara et al. (2022) and Okpara et al. (2023).

Organization demographics					Data collection				
Organization	Year founded	No. employees	Business domain	Key non-functional requirements	Number of informants	Observations	Informal conversations	Artifact analysis	Focus groups
Alpha	2013	44	Data collection and analytics	Configurability Security Scalability	5	February-May 2019	27	445	2x 3h, discussing 15 tasks
Beta	2010	56	E-commerce platform provider	Usability Performance Reliability	8	February-May 2019	42	357	2x 3h, discussing 15 tasks
Gamma	2011	36	Content provider including online advertisement management	Performance Reproducibility Configurability	5	February-May 2019	31	1,720	2x 3h, discussing 11 tasks
Delta	2013	29	Loyalty, rewards, and referral programs	Usability Maintainability Performance	11	November 2021-January 2022	34	na	na

Delta Company Delta develops loyalty, rewards, and referral products for other organizations. The products are heavily client-focused, anchored in components that are visually-rich, including integrating various 3rd party platforms and maintaining a rich dataset of client information. Key NFRs for Delta are usability, maintainability, and performance.

A.2 Data Collection

We collected data from different sources, allowing for data triangulation which can help to produce more reliable results (Lethbridge et al., 2005; Yin, 2002). We used four data collection methods (see Table 9): observation, interviews with key stakeholders, development tasks, and focus groups. Observation, interviews, and focus groups are what Lethbridge et al. (2005) classified as first-degree techniques, and analysis of documents and artifacts are so-called third-degree techniques. Whereas first-degree techniques focus on cognition, i.e., what software professionals think or feel, third-degree techniques are better suited to capture what actually happens as they represent an 'audit trail' of activity. In this study, the development artifacts provided insight into the development tasks. It also allowed us to come to a joint understanding of which NFRs were important at each company, and how a lack of shared understanding would become apparent.

A.2.1 Participant Observation

Two of the authors spent multiple days over a few months embedded in situ at each of the four organizations (see Table 11). Being present onsite allowed the investigators to become familiar with the context and staff at each organization, learn about the products on offer as well as the processes and tools used, and develop an understanding of the NFRs that were considered important at each organization (see Table 9). These initial immersive meetings and observations at the four organizations also helped us develop an understanding of how they managed shared understanding of NFRs within a CSE context. Being present onsite further afforded easy access to talk to a range of employees, spanning multiple intra-organizational boundaries, and to conduct observations and informal conversations with 47 different employees (representing roughly a third of total staff), including 23 developers, 10 development managers, five product managers, four executives, three customer success specialists, one quality assurance member, and one director of sales. We took extensive notes during the onsite observations and informal conversations. The investigators had access to a subset of employees which included at least one employee from every major team at each organization (see Table 10).

Table 10 Informants' teams interviewed at the case organizations

Organization	Teams
Alpha	Development, Dev Management, DevOps, Executive
Beta	Development, Dev Management, Product, Sales, Support, QA, Executive
Gamma	Development, Dev Management, DevOps, Executive
Delta	Development, Dev Management, Product, Support, Executive

A.2.2 Interviews

We conducted semi-structured interviews at the four organizations with staff in both development and managerial roles from these organizations. These open-ended interviews were conducted by two investigators and lasted between 30–90 minutes remotely or at the organization's office. All interviews were based on a set of base questions:

1. Are you familiar with the term DevOps? Are you familiar with the term continuous (integration, delivery, deployment)?
2. If you are familiar, how do you define these terms?
3. Does your organization practice any continuous practices or DevOps? If so, what are they?
4. Do you define NFRs?
5. How do you define an NFR?
6. Which NFRs are important to your organization?
7. How do you manage NFRs?
8. How do you document an NFR?
9. In particular, are any NFRs in your source control?
10. How do you test or ensure an NFR is satisfied?
11. How do you trace an NFR through the continuous development?
12. What happens when an NFR fails? Is there a feedback loop from continuous development?
13. Does an NFR require additional resources (additional approval, testing, etc?) when developing it?
14. How do you ensure everyone is aware of a particular NFR?

Depending on an interviewee's role or primary work, we asked additional probing questions on more specific subjects. For example, an interview with a DevOps engineer involved further questions on tools or processes used by the engineer's organization to facilitate their CSE approach. An interview with a front-end developer working extensively on the visual aesthetics of an application included questions regarding prioritizing and verification of specific NFRs (e.g., usability). All interviews were, with permission, recorded and subsequently transcribed and verified for correctness.

A.2.3 Development Tasks

We conducted an in-depth analysis of development task artifacts at Alpha, Beta, and Gamma. This analysis helped in our understanding of the shared understanding of NFRs, or lack thereof, within these three organizations. Development tasks include activities to develop new code, fix defects, document code, or test code, and these tasks typically lead to artifacts including such as code snippets, binaries, or written text. We collected information about these tasks from the organizations' source control systems; each task represented work on either a bug, feature, story, or epic, depending on the context of each organization. We collected a total of 19,060 tasks: 9,859 from Alpha, 2,629 from Beta, and 6,572 from Gamma. Of those, we selected the more recent tasks from the 12 months before the date of study (n=2,522). Selecting more recent tasks in the immediate period before the study would help informants recall details. Of the 2,522 tasks, we identified 348 as exhibiting a lack of shared understanding of NFRs. Of these 348, 174 were discussed with each respective organization through a process of member checking

Table 11 Demographics of informants

Organization	ID	Role	Experience at Organization	Total experience
Alpha	P1	Developer	2 years	20 years
Alpha	P2	Developer	10 years	20 years
Alpha	P3	Developer	10 years	20 years
Alpha	P4	Developer	5 years	10 years
Alpha	P5	Developer	10 years	20 years
Beta	P6	Developer	2 years	20 years
Beta	P7	Developer	5 years	20 years
Beta	P8	Developer	10 years	10 years
Beta	P9	Developer	5 years	5 years
Beta	P10	Developer	5 years	20 years
Beta	P11	Developer	2 years	20 years
Beta	P12	Developer	2 years	2 years
Beta	P13	Developer	2 years	5 years
Gamma	P14	Developer	2 years	2 years
Gamma	P15	Developer	2 years	20 years
Gamma	P16	Developer	2 years	5 years
Gamma	P17	Developer	5 years	20 years
Gamma	P18	Developer	2 years	5 years
Delta	P19	Visual Developer	1 year	1 year
Delta	P20	Product	5 years	5 years
Delta	P21	Visual developer	1 year	1 year
Delta	P22	Project engineer	1 year	1 year
Delta	P23	Executive	10 years	10 years
Delta	P24	Visual developer	1 year	1 year
Delta	P25	Project/Product	5 years	10 years
Delta	P26	Infrastructure	5 years	20 years
Delta	P27	Infrastructure	1 year	3 years
Delta	P28	Infrastructure	3 years	3 years
Delta	P29	Visual developer	3 years	3 years

to validate that these tasks did indeed exhibit a lack of shared understanding of an NFR. Overall, a total of 41 tasks were confirmed to exhibit a lack of shared understanding of NFRs. These tasks were used as the basis for the in-depth analysis through further focus group sessions, discussed next.

A.2.4 Focus Groups

We conducted focus groups at Alpha, Beta, and Gamma to discuss selected development artifacts that we identified as being related to a shared understanding of NFRs. Based on earlier participant observation described above, we developed a set of questions to guide the discussion of the development artifacts. A number of topics were discussed for each development task, including the factors contributing to a lack of shared understanding, which NFRs were more prone to a lack of shared understanding.

Each focus group lasted three hours and involved two investigators, one developer, and one manager. We clarified the meaning and definition of key terms, including the terms ‘non-functional requirement’ and ‘shared understanding,’ and their mutual relationship; clarification of these key terms was essential to establish a common foundation for the focus group. These sessions were, with participants’ permission, audio-recorded.

Acknowledgments We are grateful to the three anonymous reviewers and editor for their constructive feedback, which has led to a better article.

Author Contributions Colin Werner: study conceptualization, data collection, data analysis, initial draft, revisions. Laura Okpara: study conceptualization, data collection, data analysis. Klaas-Jan Stol: study conceptualization, data analysis, initial draft, revisions, Daniela Damian: study conceptualization, initial draft, revisions.

Funding This work is supported by Research Ireland (formerly Science Foundation Ireland) grant 13/RC/2094-P2 to Lero.

Data Availability All data were collected at the four organizations; we cannot make these data publicly available as the data were collected under non-disclosure agreements.

Declarations

Ethical Approval Ethical approval for the field work underpinning the theory development study presented in this paper was received from the University of Victoria, Canada.

Conflict of Interest The authors declare no conflict of interest.

Clinical trial number Clinical trial number not applicable.

Statement on the use of Generative AI The authors wish to declare that no generative AI tools were used for any step in the research process, including the writing of this article. Any resemblance of AI-generated text or style is because AI technology has learned from the best, not vice versa.

References

Ameller D, Ayala C, Cabot J, Franch X (2012a) How do software architects consider non-functional requirements: An exploratory study. In: 2012 20th IEEE International Requirements Engineering Conference (RE), pp 41–50, doi:10.1109/RE.2012.6345838

Ameller D, Ayala C, Cabot J, Franch X (2012b) Non-functional requirements in architectural decision making. IEEE Software 30(2):61–67, doi:10.1109/MS.2012.176

Antonino PO, Jung M, Morgenstern A, Faßnacht F, Bauer T, Bachorek A, Kuhn T, Nakagawa EY (2018) Enabling continuous software engineering for embedded systems architectures with virtual prototypes. In: Software Architecture: 12th European Conference on Software Architecture, Madrid, Spain, Springer, pp 115–130, doi:10.1007/978-3-030-00761-4_8

Araújo M, Araújo J, Oliveira R, Romao L, Kalinowski M (2025) Domain knowledge in requirements engineering: A systematic mapping study. In: Euromicro Conference on Software Engineering and Advanced Applications, Springer, pp 425–441, doi:10.1007/978-3-032-04200-2_29

Barbala A, Sporsem T, Stol KJ (2024) A case study of continuous adoption in the public sector in Norway. In: Hawaii International Conference on System Sciences, IEEE, doi:10.24251/HICSS.2024.248

Bass L, Clements P, Kazman R (2012) Software Architecture in Practice, 3rd edn. Addison-Wesley, Boston, Massachusetts, U.S.A.

Behutiye W, Karhapää P, Costal D, Oivo M, Franch X (2017) Non-functional Requirements Documentation in Agile Software Development: Challenges and Solution Proposal. In: Felderer M, Méndez Fernández D, Turhan B, Kalinowski M, Sarro F, Winkler D (eds) Product-Focused Software Process Improvement, Springer International Publishing, Cham, Lecture Notes in Computer Science, pp 515–522, doi:10.1007/978-3-319-69926-4_41

Behutiye W, Karhapää P, López L, Burgués X, Martínez-Fernández S, Vollmer AM, Rodríguez P, Franch X, Oivo M (2020) Management of quality requirements in agile and rapid software development: A systematic mapping study. Information and Software Technology 123:106225, doi:10.1016/j.infsof.2019.106225

Bellomo S, Ernst N, Nord RL, Ozkaya I (2014) Evolutionary improvements of cross-cutting concerns: Performance in practice. In: IEEE International Conference on Software Maintenance and Evolution, pp 545–548, doi:10.1109/ICSME.2014.91

Bellomo S, Ernst NA, Nord RL, Ozkaya I (2014) Evolutionary Improvements of Cross-Cutting Concerns: Performance in Practice. 2014 IEEE International Conference on Software Maintenance and Evolution pp 545–548, doi:10.1109/ICSME.2014.91

Bittner EAC, Leimeister JM (2014) Creating shared understanding in heterogeneous work groups: Why it matters and how to achieve it. Journal of Management Information Systems 31(1):111–144, doi:10.2753/MIS0742-1222310106

Bjarnason E, Wnuk K, Regnell B (2011) A case study on benefits and side-effects of agile practices in large-scale requirements engineering. In: Proceedings of the 1st Workshop on Agile Requirements Engineering, ACM, pp 1–5, doi:10.1145/2068783.2068786

Bjørnson FO, Dingsøyr T (2008) Knowledge management in software engineering: A systematic review of studied concepts, findings and research methods used. Information and software technology 50(11):1055–1068, doi:j.infsof.2008.03.006

Borg A, Yong A, Carlshamre P, Sandahl K (2003) The bad conscience of requirements engineering: an investigation in real-world treatment of non-functional requirements. In: Third Conference on Software Engineering Research and Practice in Sweden, pp 1–8

Bosch J (2014) Continuous Software Engineering. Springer International Publishing, Switzerland

Capilla R, Ali Babar M, Pastor O (2012) Quality requirements engineering for systems and software architecting: methods, approaches, and tools. Requirements Engineering 17:255–258, doi:10.1007/s00766-011-0137-9

Capilla R, Jansen A, Tang A, Avgeriou P, Babar MA (2016) 10 years of software architecture knowledge management: Practice and future. Journal of Systems and Software 116:191–205, doi:10.1016/j.jss.2015.08.054

Caracciolo A, Lungu MF, Nierstrasz O (2014) How do software architects specify and validate quality requirements? In: European Conference on Software Architecture, Springer, pp 374–389, doi:10.1007/978-3-319-09970-5_32

Chung L, Nixon BA, Yu E, Mylopoulos J (2000) Non-functional requirements in software engineering. Springer Science + Business Media, LLC, New York, U.S.A.

Clark HH, Brennan SE (1991) Grounding in communication. In: Resnick LB, Levine JM, Teasley SD (eds) Perspectives on Socially Shared Cognition, American Psychological Association, pp 127–149, doi:10.1037/10096-006

Cooper D, Stol KJ (2018) Adopting InnerSource: Principles and Case Studies. O'Reilly Media

Corbin JM, Strauss A (1990) Grounded theory research: Procedures, canons, and evaluative criteria. Qualitative sociology 13(1):3–21, doi:10.1007/BF00988593

Corvera Charaf M, Rosenkranz C, Holten R (2013) The emergence of shared understanding: applying functional pragmatics to study the requirements development process. Information Systems Journal 23(2):115–135, doi:10.1111/isj.12001

Creswell JW, Miller DL (2000) Determining validity in qualitative inquiry. Theory into practice 39(3):124–130, doi:10.1207/s15430421tip3903_2

Cruzes DS, Dyba T (2011) Recommended Steps for Thematic Synthesis in Software Engineering. In: 2011 International Symposium on Empirical Software Engineering and Measurement, pp 275–284, doi:10.1109/ESEM.2011.36

Dakkak A, Bosch J, Olsson HH (2022) Controlled continuous deployment: A case study from the telecommunications domain. In: Proceedings of the International Conference on Software and System Processes and International Conference on Global Software Engineering, pp 24–33, doi:10.1145/3529320.3529323

Damian D, Chisan J (2006) An empirical study of the complex relationships between requirements engineering processes and other processes that lead to payoffs in productivity, quality, and risk management. IEEE Transactions on Software Engineering 32(7):433–453, doi:10.1109/TSE.2006.61

Damian DE, Zowghi D (2002) The impact of stakeholders' geographical distribution on managing requirements in a multi-site organization. In: Proceedings IEEE Joint International Conference on Requirements Engineering, IEEE, pp 319–328, doi:10.1109/ICRE.2002.1048545

Darch P, Carusi A, Jirotka M (2009) Shared understanding of end-users' requirements in e-Science projects. In: 2009 5th IEEE International Conference on E-Science Workshops, pp 125–128, doi:10.1109/ESCIW.2009.5407963

Datadog (2025) Datadog: Cloud monitoring as a service. URL `https://www.datadoghq.com/`

Dorfman M, Thayer RH (1990) Standards, Guidelines and Examples: System and Software Requirements Engineering. IEEE Computer Society Press

Eckhardt J, Vogelsang A, Fernández DM (2016) Are "non-functional" requirements really non-functional? an investigation of non-functional requirements in practice. In: Proceedings of the 38th IEEE/ACM

International Conference on Software Engineering, pp 832–842, doi:10.1145/2884781.2884788
Eisenhardt KM (1989) Building theories from case study research. Academy of Management Review 14(4):532–550, doi:10.5465/AMR.1989.4308385
Ernst NA, Murphy GC (2012) Case studies in just-in-time requirements analysis. In: 2012 Second IEEE International Workshop on Empirical Requirements Engineering (EmpiRE), pp 25–32, doi:10.1109/EmpiRE.2012.6347678
Fitzgerald B, Stol KJ (2017) Continuous software engineering: A roadmap and agenda. Journal of Systems and Software 123:176–189, doi:10.1016/j.jss.2015.06.063
Fowler M (2024) Continuous Integration. URL https://martinfowler.com/articles/continuousIntegration.html
Fricker SA, Grau R, Zwingli A (2015) Requirements engineering: best practice. In: Requirements Engineering for Digital Health, Springer, pp 25–46, doi:10.1007/978-3-319-09798-5_2
Gacitua R, Ma L, Nuseibeh B, Piwek P, De Roeck AN, Rouncefield M, Sawyer P, Willis A, Yang H (2009) Making tacit requirements explicit. In: 2009 Second International Workshop on Managing Requirements Knowledge, IEEE, pp 40–44, doi:10.1109/MARK.2009.7
Garlan D, Tichy W, Paulisch F (1995) Software architectures (dagstuhl seminar 9508). Tech. rep., Schloss-Dagstuhl-Leibniz Zentrum für Informatik
Gawande A (2010) The Checklist Manifesto: How to Get Things Right. Penguin Books India
Geertz C (2008) Thick description: Toward an interpretive theory of culture. Routledge
Glaser B, Strauss A (1967) The Discovery of grounded theory: Strategies for qualitative research. Aldine
Glaser BG (2009) Jargonizing: The use of the grounded theory vocabulary. The Grounded Theory Review 8(1):1–16
Glinz M (2007) On non-functional requirements. In: 15th IEEE International Requirements Engineering Conference, pp 21–26, doi:10.1109/RE.2007.45
Glinz M, Fricker SA (2015) On Shared Understanding in Software Engineering: An Essay. Comput Sci 30(3-4):363–376, doi:10.1007/s00450-014-0256-x
Gralha C, Damian D, Wasserman AIT, Goulão M, Araújo J (2018) The Evolution of Requirements Practices in Software Startups. In: Proceedings of the 40th International Conference on Software Engineering, ACM, New York, NY, USA, ICSE '18, pp 823–833, doi:10.1145/3180155.3180158
Gregor S (2006) The nature of theory in information systems. MIS quarterly pp 611–642
Guba EG, Lincoln YS (1994) Competing paradigms in qualitative research. Handbook of qualitative research 2(163-194):105
Harrison NB, Avgeriou P (2007) Leveraging architecture patterns to satisfy quality attributes. In: Software Architecture: First European Conference, ECSA 2007 Aranjuez, Spain, September 24-26, 2007 Proceedings 1, Springer, pp 263–270, doi:10.1007/978-3-540-75132-8_21
Hoffmann A, Bittner EAC, Leimeister JM (2013) The emergence of mutual and shared understanding in the system development process. In: International Working Conference on Requirements Engineering: Foundation for Software Quality, Springer, pp 174–189, doi:10.1007/978-3-642-37422-7_13
Humble J, Farley D (2010) Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation (Adobe Reader). Pearson Education
Humble J, Kim G (2018) Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations. IT Revolution
ISO/IEC/IEEE (2018) IEEE/ISO/IEC 29148-2018 systems and software engineering – life cycle processes – requirements engineering
Jiang Y, Adams B (2015) Co-evolution of Infrastructure and Source Code - An Empirical Study. In: 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, pp 45–55, doi:10.1109/MSR.2015.12
Johanssen JO, Kleebaum A, Bruegge B, Paech B (2019) How do practitioners capture and utilize user feedback during continuous software engineering? In: 2019 IEEE 27th International Requirements Engineering Conference (RE), IEEE, pp 153–164, doi:10.1109/RE.2019.00023
Karhapää P, Behutiye W, Rodríguez P, Oivo M, Costal D, Franch X, Aaramaa S, Choraś M, Partanen J, Abherve A (2021) Strategies to manage quality requirements in agile software development: a multiple case study. Empirical Software Engineering 26(2):28, doi:10.1007/s10664-020-09903-x
Kautz K, Kjærgaard A (2008) Knowledge sharing in software development. PA Nielsen and K, Kautz (Eds), Software Processes & Knowledge Beyond Conventional Software Process Improvement pp 43–68
Kleinsmann M, Buijs J, Valkenburg R (2010) Understanding the complexity of knowledge integration in collaborative new product development teams: A case study. Journal of engineering and technology management 27(1-2):20–32, doi:10.1016/j.jengtecman.2010.03.003
Klotins E, Gorschek T, Wilson M (2023) Continuous software engineering: Introducing an industry readiness model. IEEE Software 40(4):77–87, doi:10.1109/MS.2022.3238633

Landis JR, Koch GG (1977) The measurement of observer agreement for categorical data. Biometrics 33(1):159–174, URL https://pubmed.ncbi.nlm.nih.gov/843571/

Latour B (1986) The powers of association. In: Law J (ed) Power, Action and Belief: A New Sociology of Knowledge?, Routledge & Kegan Paul, p 264–80

Lethbridge TC, Sim SE, Singer J (2005) Studying software engineers: Data collection techniques for software field studies. Empirical software engineering 10:311–341, doi:10.1007/s10664-005-1290-x

Lincoln Y, Guba E (1985) Naturalistic inquiry. Sage

Lundberg L, Bosch J, Häggander D, Bengtsson PO (1999) Quality attributes in software architecture design. In: Proceedings of the IASTED 3rd International Conference on Software Engineering and Applications, pp 353–362

Mairiza D, Zowghi D, Nurmuliani N (2010) An investigation into the notion of non-functional requirements. In: Proceedings of the 2010 ACM symposium on applied computing, pp 311–317, doi:10.1145/1774088.1774142

Marshall C, Rossman GB (2014) Designing qualitative research. Sage publications

Mattos DI, Bosch J, Olsson HH (2018) Challenges and strategies for undertaking continuous experimentation to embedded systems: Industry and research perspectives. In: Agile Processes in Software Engineering and Extreme Programming, pp 277–292, doi:10.1007/978-3-319-91602-6_20

Méndez Fernández D (2018) Supporting requirements-engineering research that industry needs: The NaPiRE initiative. IEEE Software 35(1):112–116, doi:10.1109/ms.2017.4541045

Méndez Fernández D, Wagner S, Kalinowski M, Felderer M, Mafra P, Vetrò A, Conte T, Christiansson MT, Greer D, Lassenius C, Männistö T, Nayabi M, Oivo M, Penzenstadler B, Pfahl D, Prikladnicki R, Ruhe G, Schekelmann A, Sen S, Spinola M, Tuzcu A, de la Vara JL, Wieringa R (2017) Naming the pain in requirements engineering: contemporary problems, causes, and effects in practice. Empirical Software Engineering 22(5):2298–2338, doi:10.1007/s10664-016-9451-7

Meyer AN, Barr ET, Bird C, Zimmermann T (2021) Today was a good day: The daily life of software developers. IEEE Transactions on Software Engineering 47(5):863–880, doi:10.1109/TSE.2019.2904957

Meyer B (2013) The abc of software engineering. URL https://bertrandmeyer.com/2013/03/25/the-abc-of-software-engineering/

Miles MB, Huberman AM (1994) Qualitative data analysis: An expanded sourcebook. Sage

Miner JB (1980) Theories of Organizational Behavior. The Dryden Press

Miner JB (2015) Organizational behavior 1: Essential theories of motivation and leadership. Routledge

Mintzberg H (2017) Developing theory about the development of theory. In: Handbook of middle management strategy process research, Edward Elgar Publishing, pp 177–196, doi:10.1093/oso/9780199276813.003.0017

Nasir S, Guerra E, Zaina L, Melegati J (2023) An exploratory study about non-functional requirements documentation practices in agile teams. In: Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing, pp 1009–1017, doi:10.1145/3555776.3577605

Niknafs A, Berry D (2017) The impact of domain knowledge on the effectiveness of requirements engineering activities. Empirical Software Engineering 22:80–133, doi:10.1007/s10664-015-9416-2

Okpara L, Werner C, Murray A, Damian D (2022) A case study of building shared understanding of non-functional requirements in a remote software organization. In: 2022 IEEE 30th International Requirements Engineering Conference (RE), doi:10.1109/RE54965.2022.00008

Okpara L, Werner C, Murray A, Damian D (2023) The role of informal communication in building shared understanding of non-functional requirements in remote continuous software engineering. Requirements Engineering 28:595–617, doi:10.1007/s00766-023-00404-z

Olsson HH, Bosch J (2014) Towards agile and beyond: an empirical account on the challenges involved when advancing software development practices. In: International Conference on Agile Software Development, Springer, pp 327–335, doi:10.1007/978-3-319-06862-6_27

Pohl K, Rupp C (2016) Requirements engineering fundamentals: a study guide for the certified professional for requirements engineering exam-foundation level-IREB compliant. Rocky Nook, Inc.

Popper K (1980) The Logic of Scientific Discovery. Unwin Hyman

Quin F, Weyns D, Galster M, Silva CC (2024) A/b testing: A systematic literature review. Journal of Systems and Software 211(112011), doi:10.1016/j.jss.2024.112011

Rahy S, Bass JM (2022) Managing non-functional requirements in agile software development. IET Software 16(1):60–72, doi:10.1049/sfw2.12037

Ralph P (2019) Toward methodological guidelines for process theories and taxonomies in software engineering. IEEE Transactions on Software Engineering 45(7):712–735, doi:10.1109/TSE.2018.2796554

Ramesh B, Cao L, Baskerville R (2010) Agile requirements engineering practices and challenges: an empirical study. Information Systems Journal 20(5):449–480, doi:10.1111/j.1365-2575.2007.00259.x

Richardson I, Von Wangenheim CG (2007) Guest editors' introduction: Why are small software organizations different? IEEE Software 24(1):18–22, doi:10.1109/MS.2007.12

Ries E (2011) The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses, 1st edn. Currency, New York

Robson C (2002) Real world research, 2nd edn. Blackwell Publishers

Ryan S, O'Connor RV (2009) Development of a team measure for tacit knowledge in software development teams. Journal of Systems and Software 82(2):229–240, doi:10.1016/j.jss.2008.08.021

Ryan S, O'Connor RV (2013) Acquiring and sharing tacit knowledge in software development teams: An empirical study. Information and Software Technology 55(9):1614–1624, doi:10.1016/j.infsof.2013.02.013

Sánchez-Gordón ML, O'Connor RV (2016) Understanding the gap between software process practices and actual practice in very small companies. Software Quality Journal 24:549–570, doi:10.1007/s11219-015-9282-6

Sandberg J, Alvesson M (2021) Meanings of theory: Clarifying theory through typification. Journal of Management Studies 58(2):487–516, doi:10.1111/joms.12587

Savor T, Douglas M, Gentili M, Williams L, Beck K, Stumm M (2016) Continuous deployment at Facebook and OANDA. In: Proceedings of the 38th IEEE/ACM International Conference on Software Engineering Companion, ACM Press, Austin, Texas, pp 21–30, doi:10.1145/2889160.2889223

Schön EM, Thomaschewski J, Escalona MJ (2017) Agile Requirements Engineering: A systematic literature review. Computer Standards & Interfaces 49:79–91, doi:10.1016/j.csi.2016.08.011

Seaman CB (1999) Qualitative methods in empirical studies of software engineering. IEEE Transactions on Software Engineering 25(4):557–572, doi:10.1109/32.799955

Shahin M, Babar MA, Zhu L (2017) Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices. IEEE Access 5:3909–3943, doi:10.1109/ACCESS.2017.2651689

Sharma GG, Stol KJ (2020) Exploring onboarding success, organizational fit, and turnover intention of software professionals. Journal of Systems and Software 159:1–16, doi:10.1016/j.jss.2019.110442

Sporsem T, Hatling M, Tkalich A, Stol KJ (2023) Knowns and unknowns: An experience report on discovering tacit knowledge of maritime surveyors. In: International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ), Springer, doi:10.1007/978-3-031-29786-1_22

Stol K, Babar MA, Avgeriou P, Fitzgerald B (2011) A comparative study of challenges in integrating open source software and inner source software. Information and Software Technology 53(12):1319–1336, doi:10.1016/j.infsof.2011.06.007

Stol KJ (2024) Teaching theorizing in software engineering research. In: Mendez D, Avgeriou P, Kalinowski M, bin Ali N (eds) Teaching Empirical Research Methods in Software Engineering, Springer, doi:10.1007/978-3-031-71769-7_3

Stol KJ, Fitzgerald B (2018) The ABC of software engineering research. ACM Transactions on Software Engineering and Methodology 27(3):11, doi:10.1145/3241743

Stol KJ, Avgeriou P, Babar MA, Lucas Y, Fitzgerald B (2014) Key factors for adopting inner source. ACM Transactions on Software Engineering and Methodology 23(2), doi:10.1145/2533685

Stol KJ, Ralph P, Fitzgerald B (2016) Grounded theory in software engineering research: a critical review and guidelines. In: 2016 IEEE/ACM 38th International Conference on Software Engineering, IEEE, pp 120–131, doi:10.1145/2884781.2884833

Stol KJ, Schaarschmidt M, Goldblit S (2022) Gamification in software engineering: The mediating role of developer engagement and job satisfaction. Empirical Software Engineering 27, doi:10.1007/s10664-021-10062-w

Strauss A, Corbin J (1990) Basics of Qualitative Research. Sage

Stray V, Hanssen GK, Barbala A, Smite D, Stol KJ (2025) What is generative AI good for? introduction to the special issue on generative AI in software engineering. Information and Software Technology 187, doi:10.1016/j.infsof.2025.107857

Svensson RB, Höst M, Regnell B (2010) Managing quality requirements: A systematic review. In: 2010 36th EUROMICRO conference on software engineering and advanced applications, IEEE, pp 261–268, doi:10.1109/SEAA.2010.55

Tkalich A, Klotins E, Sporsem T, Stray V, Moe NB, Barbala A (2025) User feedback in continuous software engineering: revealing the state-of-practice. Empirical Software Engineering 30(3):79, doi:10.1007/s10664-024-10557-2

Wagner S, Méndez Fernández D, Felderer M, Vetrò A, Kalinowski M, Wieringa R, Pfahl D, Conte T, Christiansson MT, Greer D, Lassenius C, Männistö, Nyebi M, Oivo M, Penzenstadler B (2019) Status quo in requirements engineering: A theory and a global family of surveys. ACM Transactions on Software Engineering and Methodology 28(2):1–48, doi:10.1145/3306607

Werner C, Li ZS, Ernst N, Damian D (2020) The lack of shared understanding of non-functional requirements in continuous software engineering: Accidental or essential? In: 2020 IEEE 28th International Requirements Engineering Conference (RE), pp 90–101, doi:10.1109/RE48521.2020.00021

Werner C, Li ZS, Lowlind D, Elazhary O, Ernst N, Damian D (2022) Continuously managing NFRs: Opportunities and challenges in practice. IEEE Transactions on Software Engineering 48(7):2629–2642, doi:10.1109/TSE.2021.3066330

Wiegers K, Beatty J (2013) Software Requirements, 3rd edn. Microsoft Press, Redmong, Washington, U.S.A.

Williams D, Williams D (2009) Exploring software project characteristics on scope creep. In: Proceedings of Southern Association for Information Systems (SAIS), pp 39–44

Williams L, Cockburn A (2003) Agile software development: It's about feedback and change. Computer 36(6):39–43, doi:10.1109/MC.2003.1204373

Yin RK (2002) Case Study Research: Design and Methods, 3rd edn. SAGE Publications, Inc, Thousand Oaks, Calif